Manejo de Ficheros

Unidad 1

Tabla de contenido

Introducción **5**

¿Qué es un fichero? **7**

¿Cómo es un fichero por dentro? 8
Codificaciones de caracteres 9
Sistema de Ficheros 11
Tipos de ficheros 12
Rutas 14
Extensión de un fichero 16
Permisos sobre ficheros 16
Operaciones con ficheros 19

Formas de acceso a un fichero **21**

Secuencial 21
Aleatorio 22
Ventajas e inconvenientes de las distintas formas de acceso 23
Clases Java de acceso a un fichero 23

Clases asociadas a operaciones de gestión de ficheros y directorios **24**

Librerías específicas utilizadas en la gestión de ficheros 25
1. java.io (Java Input Output) 26
   - ❖ Clase File (Java.io.File) 27
   - ❖ Manejo de ficheros en Java con la clase File 29
   - ❖ Constantes de la clase File 31
   - ❖ Métodos de la clase File 32
   - ❖ Interface FilenameFilter: Creación de un filtro 33
2. java.nio (Java Non-blocking Input Output) 35
   - ❖ Métodos de la clase PATH 36
   - ❖ Métodos de la clase Files 37

Ejemplo listado de archivos 38
Ejemplo escribir bytes 38
Ejemplo leer bytes 38
Ejemplo escribir caracteres 39
Ejemplo copia fichero 39
Ejemplo mover fichero 39
Operaciones básicas de ficheros 40
Creación de un fichero 41
   - ❖ Utilizando java.io.file 41
   - ❖ Utilizando java.nio.file.Files 42

Creación de un directorio 43
   - ❖ Utilizando java.io.file 43
   - ❖ Utilizando java.nio.file.Files 44

Borrado de un fichero 45
   - ❖ Utilizando java.io.file 45
   - ❖ Utilizando java.nio.file.Files 46

Copia 47
   - ❖ Utilizando java.nio.file.File 47
   - ❖ Utilizando java.io.file 47

Movimiento 49
   - ❖ Utilizando java.nio.file.Files 49

❖ Utilizando java.io.file....49

EJERCICIOS BLOQUE 1 50

Gestión de flujos de datos: Streams – Acceso secuencial 51

Streams binarios o de bytes 53
Constructores InputStream 54
Constructores OutputStream 55
Ejemplo convertir un objeto en un fichero 56
Ejemplo convertir un fichero en un objeto 58
Streams de texto 61
Constructores FileWriter 61
Constructores FileReader 61
Ejemplo escribir en un fichero 63
Ejemplo leer de un fichero 63

Gestión de flujos de datos - Streams – Acceso Aleatorio 64

Constructores RandomAccessFile 64
Ejemplo RandomAccesFile: 65

EJERCICIOS FICHEROS SECUENCIALES 66

EJERCICIOS FICHEROS ALEATORIOS 67

EJERCICIOS BLOQUE 2 69

Ficheros XML 71

¿Qué es un fichero XML? 71
¿Cómo están formados? 73
Etiquetas XML 73
Atributos XML 73
¿Cómo leer ficheros XML? 74
DOM (Document Object Model) 75
Leer Archivos - DOM 76
Escribir Archivos – DOM 79
SAX (API Simple para XML) 83
Leer Archivos - SAX 83
Binding 86
Configuración con anotaciones 86
Ejemplo de anotaciones 87
Funcionamiento de JAXB 90
❖ Marshall 91
❖ UnMarshall 92
❖ Ejemplo: Marshall, Unmarshall 92

Librerías para la conversión de documentos XML a otros formatos 94

Introducción a JasperReport 95
Cómo instalar Jaspersoft Studio 6.21.3 96
Diseñar y compilar la plantilla 98
Rellenar el informe con datos, exportar informe 103

EJERCICIOS BLOQUE 3 104

Documentación de las aplicaciones desarrolladas 105

PRÁCTICA FINAL 1 108

PRÁCTICA FINAL 2 109

Control de excepciones 110

Sentencia try-catch-finally 111

Debido a la importancia de los ficheros en el acceso a datos, en este tema aprenderemos conceptos básicos que hay que tener en cuenta a la hora de trabajar con ficheros.

Introducción

Si estás estudiando este módulo, es probable que ya hayas estudiado el de programación, por lo que no te serán desconocidos muchos conceptos que se tratan en este tema.

Ya sabes que cuando apagas el ordenador, los datos de la memoria RAM se pierden. Un ordenador utiliza ficheros para guardar los datos. Piensa que los datos de las canciones que oyes en mp3, las películas que ves en formato avi, o mp4, etc., están, al fin y al cabo, almacenadas en ficheros, los cuales están grabados en un soporte como son los discos duros, DVD, pendrives, etc.

Se llama a los datos que se guardan en ficheros **datos persistentes**, porque persisten más allá de la ejecución de la aplicación que los trata. Los ordenadores almacenan los ficheros en unidades de almacenamiento secundario como discos duros, discos ópticos, etc. En esta unidad veremos, entre otras cosas, cómo hacer con Java las operaciones de crear, actualizar y procesar ficheros.

A las operaciones, que constituyen un flujo de información del programa con el exterior, se les conoce como Entrada/Salida (E/S).

Las operaciones de E/S en Java las proporciona el paquete estándar de la API de Java denominado java.io que incorpora interfaces, clases y excepciones para acceder a todo tipo de ficheros.

La librería java.io contiene las clases necesarias para gestionar las operaciones de entrada y salida con Java. Estas clases de E/S las podemos agrupar fundamentalmente en:

- Clases para leer entradas desde un flujo de datos.
- Clases para escribir entradas a un flujo de datos.
- Clases para operar con ficheros en el sistema de ficheros local.
- Clases para gestionar la serialización de objetos.

```
java.io
 ├── Leer datos (Input)
 │    └── InputStream, Reader, BufferedReader
 ├── Escribir datos (Output)
 │    └── OutputStream, Writer, PrintWriter
 ├── Manejar archivos
 │    └── File
 └── Guardar/recuperar objetos (Serialización)
      └── ObjectOutputStream, ObjectInputStream
```

java.nio (Non- Blocking I/O) fue introducido en el API de Java desde la versión 1.4 como extensión eficiente a los paquetes java.io y java.net. Java NIO ofrece una forma diferente de trabajar con IO que las API de IO estándar. Se basa en el "Buffer" y el "Channel".

java.io vs java.nio:

- Depende de lo que necesitemos pueden ser complementarios y utilizaremos conjuntamente ambos paquetes.
- java.nio permite manejar múltiples canales (archivos o conexiones de red) con uno o unos pocos hilos.
- En java.nio el procesamiento de datos es más complicado que usar los streams bloqueantes de java.io
- java.nio es la opción si necesito manejar cientos de conexiones (canales) abiertas y en cada una manejar una pequeña cantidad de datos.
- java.io es la opción si voy a manejar pocas conexiones con un alto ancho de banda (envío mucha información a la vez)
- Hasta JSE7 java.io.File era la clase utilizada para realizar operaciones I/O con archivos
- Esta clase tenía limitaciones como:

  ✓ Muchos métodos no lanzaban excepciones por lo que era muy complicado detectar y resolver errores
  ✓ No permitía manejar enlaces simbólicos
  ✓ Presenta problemas de escalabilidad. Directorios largos provocaban problemas de memoria e incluso de denegación de servicio

- El paquete java.nio.file incorporado a partir de JSE7 resuelve estos problemas.
- Es el que debemos usar para trabajar con archivos independientemente de si realizamos I/O con streams (java.io) ó con buffers y channels (java.nio)

Indica si la afirmación es verdadera o falsa:
Los datos persistentes perduran tras finalizar la ejecución de la aplicación que los trata.
a. Verdadero
b. Falso

¿Qué es un fichero?

Un ordenador, lo único que se es capaz de manejar es un conjunto de bits (0, 1). Estos bits se agrupan en la unidad mínima, el byte. Un byte equivale a 8 bits.

Un fichero (File en inglés) es una archivo que contendrá un conjunto de caracteres o bytes que se almacenarán en un dispositivo en una ruta y con un nombre y extensión concretos.



En Java se sigue el concepto de un fichero como un conjunto de bytes.

Los ficheros suelen llevar asociado una ruta de acceso, un nombre, y una extensión para saber de que tipo de fichero se trata. El conjunto de los 3 elementos debe ser único.

Los ficheros se guardan en dispositivos de almacenamiento fijo como discos duros, pendrives, tarjetas, CD, etc… Por esta razón los ficheros NO son VOLÁTILES (son PERSISTENTES).

La manera en que se organizan los datos dentro de un fichero depende únicamente del diseñador de ese tipo de archivo.

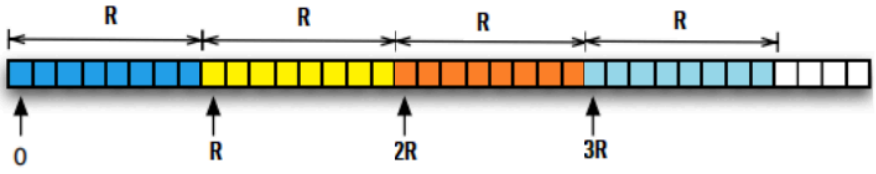
NOTA: Un archivo y un fichero en el contexto de la informática son exactamente lo mismo.



¿Cómo es un fichero por dentro?

Se puede decir que no existe una forma predeterminada de cómo es la estructura de un fichero ya que cada desarrollador puede diseñarlo como quiera.

Lo importante es que estará formado por registros.
Los registros son bloques de bytes que guardan la información deseada.



¿Qué son los registros?
Se puede llamar registro a cada bloque de longitud fija por el que está formado un fichero.

Gracias a la extensión y el software correspondiente el SO puede interpretar el fichero y mostrarlo de forma correcta.

NOTA: Intenta abrir una imagen en formato texto y comprobarás que no es legible.

```
?˙¶Aç_X₁‘?U∂jJt¤U÷I5®El‘/˛4A¥Ak˘é_õ?≤rfl˜P˘$QEyó=0¢ä(∏QE¿(¢ä.EQp
(¢ãÄQE\ä(¢‡Q\≈Î^¯
ØÎö*@//˜õiη]—Gqsq¨R:˘£Ã¨Gp∏Ô])TƒWÖöK‘ª#
ı°á£*”/)∑Ëμ;ÏR‡˙Wμi˘∞áÉNìm˘      Δ/ãzfi≥Âè∂_C„y¸çÛL|Ã∞@ãJNHE2z‘œ¯˙˘ÜøÙS˛/˘· º Ø÷Wá¯€k^?s?(~
˙?ÁÃøÛ<‘”åJ˜o¯˙˘ÜøÙS˛/· º£˛?·Ø˘˘ã˘¯rØ)˘ƒ?Δ˘œ¯˘Ã?◊˛˛¯ófiøÃ◂úJLJ˜¯˙˘ÜøÙS˛/˘· º§?∞√\qÒ;˛˘˛´ ?
˛„Á˛~Ê/ı˘˘>%˜ØÛ<‘Œï„ρá◊ü˘l=·Õóå1œx7≈º÷4ıÔ=˛fi_ÈWWÒKπ
≠,2−‹dÀ+#|ƒ6≠~sõÂïÚ|k¬÷iª‘u≥L˝*ÃËƒ¯%ä¢öMμg∫k−(¢ä◂Ó{AEQp
(¢ãÄQE\ä(¢‡QE¢ä(∏QE¿(¢ä.EQp
À◊?&G◊?Ï?˛äj‘¨Qw˛D]w˛i∑˙)
´ZO˜±13©˛9z3Ï˛=®¶Q_€'Ôuºœöe˘êŒ≥˘a≠C˘Kf¶”•˘êŒ≥˘a≠C˘Kf¶◊Ò¶g˘#˘„óÊaÂfIÚ.
°˛˛AEWîzAEPEPEPEPEPEPEP^EÒ◊˛MüQ˜∞fiã˜ßã:ı/Ú/éøÚl˙ó”ÜÙ_˝<Y◊∑ì˘»fl
˘_!˘•#«Ðø%Uà˘ø&~ÿ§R“QK_ÿßÒ”
◂]7,ÜnÆ˘iC˛M◂õE±≥õ¬^◂}Æß,}U˜õàμ+ÈÿYŸ¶P>œ”>A8x±åö^ÕSR≤—º9´ÍWQŸiəVÔqwq)¬E)gr{†ü¬œY˝é¥
€ÍK^s◊æ3Îə≤[xó,Ôâ.|e4sèfi[ÿMə.‹ûΠé¬^;oZ}iEPÁÍ\„e_˝SøÉfï\MvflμwˇsØ‡á˝ìøÉfï\M4Ò◊˛è?
Ì»˛ßÙØ˘»è˛fióËQE~j~éQEQEQEQEQEQEQEQEóÆ˘»ãÆ˘ÿ6„˘E5jV^ª˘”.ª˘`€è˝’≠/,«‘Œß◂•Ëœ´(¢ä˛€?
ãœöe˝êŒ≥˘a≠C˘Kf¶”•˘êŒ≥˘a≠C˘Kf¶◊Ò¶g˘#˘„óÊaeflÚ.
°˛˛AEWîzAEPEPEPEPEPEPEP^EÒ◊˛MüR˘∞fiã˘ßã:ı/ÛÔãæ’|W◂ƒ6ÇêÔÆf/ÛOÜ,O.
9À∫ʃãË"f˛ÍM›∑fΩlÆ§)ft*MŸFqm˜&è/3ÑÍÂ’°v·$óûô°(›ı−|Ee°y|&m&Ÿıè˛P—μVåÕ9˛ÍW−k&>hÃ∞DÒ>F&fS’≥˘
„◂W˛ÔÔãfl˜k5è˛1_◊ÅÕj˛”?z?ê~•çZ:R˘¿_˜ıÒ”·Œ©Ôw˘TÒá√−+ƒß¬?
◂í[&ü®jinft≤íEq¢Ù\<êy±œ d›ÉåM”¥Î'A≤”4€XÏ¥°;t∑ə∑âpëFä¿˙W«^Ò¯*H√ø…=˛*Õc˘åV~ù˘¯¨YKs£ÿ|
D'm,nί/Ym>Í“™Mmí2V
VeÏ©bpÕ]M}ËóÑ≈'gNW~L°íä¯Ø˛«‡Ø˝
flø◂÷k˛bè¯oÇá˛eÔâfl¯k5è˛1S₁º/˛˛èñiä˙ñ3˛}K˝&yßÌ_˘),¯˘dÔfü˙Y•WYû>¯è«o€?√_<7·ù√˛◂ØÑØ¥àò|K§…
¶<jów∑“øïm.%Dñä
é´π§¬Ç5ß˛Õºi^ç|ÌʃrRJ)ifl’−‹Bμ
í*¨\[íùùÂÆWAßËEPEPEPEPEPEPEPEPEPYzÔ˛àʃÔ˝Én?ÙSV•eÎøÚ"Îø^
∏˘−MZ”˛˛}ŁÍ
^å˙≤ä(ØÍ≥¯º¯¶˛¯                                                                    +
```

Codificaciones de caracteres

Las codificaciones de archivos, también denominadas codificaciones de caracteres, especifican cómo se representan los caracteres durante el procesamiento de texto. Una codificación puede ser preferible sobre otra por lo que se refiere a los caracteres del lenguaje que puede o no controlar, aunque normalmente se prefiere Unicode.

Cuando se leen o escriben archivos, la correspondencia incorrecta de las codificaciones de archivo puede producir excepciones o resultados incorrectos. Los caracteres no son legibles y no podemos interpretar la información. Este sería un ejemplo de mala interpretación de un encoding:

H‰ÄTMoÓ@1⁄4ī xGûàõ~z×¨aŐ ©¨(8¤=Ç)AÔn£Â¿ç1⁄2õ¦IH‘ƒ3Ÿ3ã

Se denomina ***encoding*** al sistema utilizado para transformar los caracteres que usa cada lenguaje en un símbolo que un ordenador puede interpretar.

Para poder hacer la equivalencia entre byte y símbolo (número, letra, símbolo especial, etc.) se utiliza la tabla ASCII.

La tabla ASCII permite hasta 128 símbolos, ver tabla ASCII en https://www.ascii-code.com/es ó https://elcodigoascii.com.ar

Existen varios tipos de codificaciones de caracteres:

- ISO-8859: Este tipo de encoding es bastante conocido y se caracteriza por incluir letras, símbolos y caracteres, pero además también los acentos y los símbolos de interrogación y exclamación. Este tipo de codificación utiliza 8 bits, por tanto, tiene una capacidad de 256 caracteres, lo que la hace más amplia que ASCII. Incluye los 128 caracteres de ASCII, pero se añaden símbolos matemáticos y letras griegas, entre otros. Con el tiempo, se han ido creando diferentes especializaciones de esta codificación. Podemos encontrar también estos otros:
  - ✓ ISO 8859-1 (Latin-1), para la zona de Europa occidental.
  - ✓ ISO 8859-2 (Latin-2), para la zona de Europa occidental y Centroeuropa.
  - ✓ ISO 8859-3 (Latin-3), para la zona de Europa occidental y Europa del sur.
  - ✓ ISO 8859-4 (Latin-4), para la zona de Europa occidental y países bálticos (lituano, estonio y lapón).
  - ✓ ISO 8859-5, para el alfabeto cirílico.
  - ✓ ISO 8859-6, para el alfabeto árabe.
  - ✓ ISO 8859-7, para el alfabeto griego.
  - ✓ ISO 8859-8, para el alfabeto hebreo.

- ✓ ISO 8859-9 (Latin-5), para la zona de Europa occidental con los caracteres del alfabeto turco.
- ✓ ISO 8859-10 (Latin-6), para la zona de Europa occidental, incluye los caracteres del alfabeto nórdico, lapón y esquimal.
- ✓ ISO 8859-11, incorpora caracteres del alfabeto tailandés.
- ✓ ISO 8859-13 (Latin-7), incorpora caracteres para los idiomas bálticos y el polaco.
- ✓ ISO 8859-14 (Latin-8), incorpora caracteres para los idiomas celtas.
- ✓ ISO 8859-15 (Latin-9), añade el símbolo del euro.
- ✓ ISO 8859-16, incorpora caracteres para los idiomas polaco, checo, eslovaco, húngaro, albano, rumano, alemán e italiano.

- Unicode: Es una norma de codificación creada en 1991 para unificar los tipos de codificación. Como hemos visto, existen multitud de variantes de codificación para abarcar diferentes idiomas. La creación de este encoding pretendía organizar en un mismo estándar los diferentes caracteres dentro de una misma codificación, para poder abarcar diferentes idiomas tanto de alfabetos europeos como de chinos, japoneses, coreanos o lenguas ya extinguidas con alfabetos diferentes. Para realizar las tablas de equivalencias, "Unicode" asigna un identificador numérico a cada carácter, pero también irá acompañado de información como la direccionalidad, la capitalización y otros atributos. Nuestro ordenador, según su arquitectura, utilizará diferentes bloques de 8, 16 o 32 bits para interpretar y representar los números. Estos tres diferentes bloques han creado diferentes codificaciones:
  - ✓ UTF-8.
  - ✓ UTF-16.
  - ✓ UTF-32.

Hoy en día, la codificación más usada es la codificación de caracteres UTF-8.

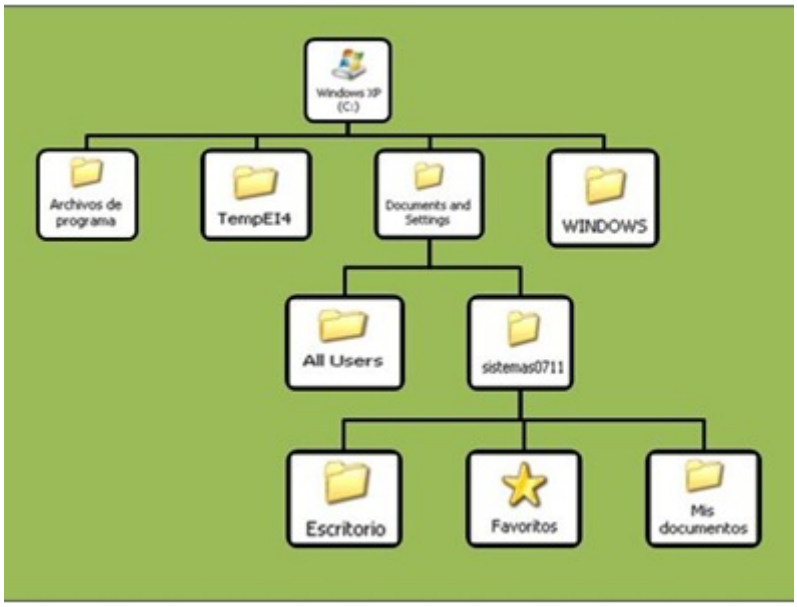
Tanto los editores de texto, como los IDE (programa para desarrollar nuestra aplicación) normalmente dan la posibilidad de configurar qué tipo de codificación queremos usar.

En el caso de que queramos usar diferentes encodings, será necesario usar un fichero binario, es decir, un fichero que almacenará bytes con la información.

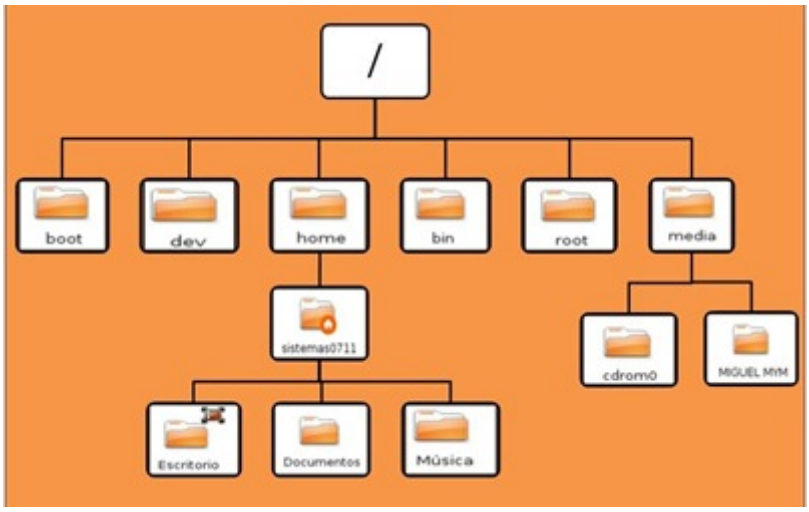
Sistema de Ficheros

Los ficheros se suelen organizar de forma jerárquica.

- Esta imagen muestra el árbol de archivos en Windows XP:



- Este otra imagen muestra el árbol de ficheros en Linux :



Tipos de ficheros

Existen diferentes tipos de ficheros:

-  Fichero estándar: es un archivo que contiene todo tipo de datos: caracteres, imagen, audio, vídeo, etcétera. Normalmente son ficheros que contienen información de cualquier tipo.
-  Directorios o carpetas: son ficheros que albergan más archivos en su interior. Su principal utilidad es mantener un orden o jerarquía en nuestros sistemas.
-  Ficheros especiales: son todos esos archivos que usa nuestro sistema operativo y que se utilizan para controlar los dispositivos o periféricos de nuestro ordenador.

En este tema profundizaremos en el tipo de ficheros estándar y en los directorios. Como explicaremos más adelante, este tipo de ficheros nos permitirán realizar diferentes acciones para tratar los ficheros y para mantener un orden y jerarquía con las carpetas.

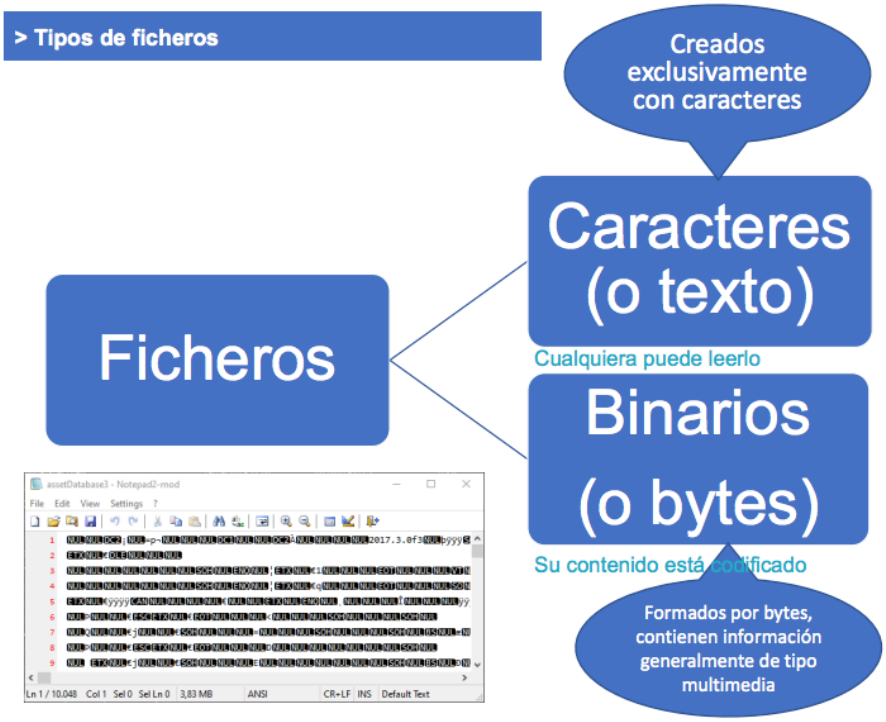
- TIPOS DE FICHEROS
  - FICHERO ESTÁNDAR
    - Válido para todo tipo de datos (caracteres, imágenes, vídeo, etc)
  - DIRECTORIO O CARPETA
    - Fichero que almacena más ficheros en su interior (jerarquía)
  - FICHEROS ESPECIALES
    - Del SO y se usan para controlar dispositivos o periféricos



Podemos destacar dos tipos de ficheros de datos:

- Los ficheros de bytes: también conocidos como ficheros binarios, son archivos que usan los programas para leer o escribir información.
- Los ficheros de caracteres: también conocidos como ficheros de texto, nos permitirán leer o escribir la información que contengan.

Java puede operar con ficheros de cualquier tipo.



Podemos destacar dos tipos de acceso a los ficheros de datos, que analizaremos más tarde:

- Acceso secuencial
- Acceso aleatorio

Rutas

- Para acceder a un determinado fichero, se utiliza la ruta (path). Una ruta indica la dirección del fichero en el sistema de archivos.
- En una ruta, cada nivel de la jerarquía se representa delimitado por el símbolo /. En Windows, el símbolo separador es \ .
- Además de /, existen 2 elementos especiales en la ruta:
  - ✓ . Un punto representa al directorio actual.
  - ✓ .. Dos puntos representan al directorio padre en la jerarquía (subir un nivel).

- Existen 2 tipos de rutas:
  - ✓ Absoluta: representa la ruta completa del recurso. Una ruta absoluta indica la ubicación completa de un archivo o carpeta desde la raíz del sistema.

Ejemplos de rutas absolutas en Linux:
C:/Users/Juan/Documents/reporte.txt
D:/Musica/rock/cancion.mp3
C:/Program Files/Java/jdk-17/bin/java.exe

Ejemplos de rutas absolutas en Windows:
C:\Users\Juan\Documents\reporte.txt
D:\Musica\rock\cancion.mp3
C:\Program Files\Java\jdk-17\bin\java.exe

Estas rutas **siempre empiezan con una letra de unidad** (C:, D:, etc.) y son **independientes del lugar donde estés ubicado** en el sistema de archivos.

  - ✓ Relativa: representa solo una parte de la ruta, ya que en ella se tiene en cuenta el directorio actual desde el que se está trabajando. Por lo tanto, una ruta relativa indica la ubicación de un archivo o carpeta relativa a tu ubicación actual (el directorio actual, también llamado working directory).

Si asumimos que estamos en **C:\Users\Juan**

Entonces estas rutas relativas serían en Windows:

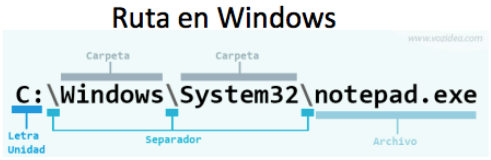
Ruta relativa	Equivale a ruta absoluta
Documents\reporte.txt	C:\Users\Juan\Documents\reporte.txt
..\Publico\info.txt	C:\Users\Publico\info.txt
.\musica.mp3	C:\Users\Juan\musica.mp3

Entonces estas rutas relativas serían en Linux:

Ruta relativa	Equivale a ruta absoluta
Documents/reporte.txt	C:/Users/Juan/Documents/reporte.txt
../Publico/info.txt	C:/Users/Publico/info.txt
./musica.mp3	C:/Users/Juan/musica.mp3

En Java para trabajar con ficheros, se utiliza la ruta relativa (“ficheros/persona.dat”) parte de la ubicación del fichero ejecutable. Aunque también se permiten rutas absolutas (“c:/ ficheros/persona.dat”) pero a la hora de distribuir la aplicación las rutas absolutas NO son prácticas.

- La ruta indica la ubicación de ese fichero en nuestro sistema
- Formada por diferentes niveles jerárquicos (carpetas) separado por un símbolo barra /,
  - Windows, se utiliza la contrabarra \
  - Unix el separador es /
- Eclipse admite tanto / como \ cuando definimos la ruta.





En la imagen está mal, en Linux se usa / no \. Aunque ya hay versiones que admiten \.

Ejemplo en Java (Windows)

```
import java.io.File;


public class RutaEjemplo {
    public static void main(String[] args) {
        // Ruta absoluta
        File archivoAbsoluto = new File("C:\\Users\\Juan\\Documents\\reporte.txt");
        System.out.println("Ruta absoluta: " + archivoAbsoluto.getAbsolutePath());


        // Ruta relativa
        File archivoRelativo = new File("Documents\\reporte.txt");
        System.out.println("Ruta relativa convertida: " + archivoRelativo.getAbsolutePath());
    }
}
```

Si ejecutas esto desde **C:\Users\Juan\,** ambos archivos apuntarían al mismo lugar.

Extensión de un fichero

La extensión sirve para asociar un determinado fichero con un programa/s que lo ejecuten o interpreten.

- Indica cómo debe ser interpretado el fichero.
  - ✓ doc, docx, odt 
  - ✓ html, xml, jsp 
  - ✓ mp3, avi, wma 

- Se considera extensión lo existente a partir del último punto del nombre del archivo:
  - ✓ HolaMundo.doc -> extensión =doc
  - ✓ HolaMundo.doc.mp3 -> extensión= mp3

Permisos sobre ficheros

El usuario que crea el fichero tiene derecho a decidir quien y cómo accede a su fichero.

Existen 3 grupos para los que se les puede definir permisos:

- u. Propietario
- g. Grupo
- o. Resto de usuarios

Los permisos que se pueden dar son los siguientes:

- r. Lectura
- w. Escritura
- x. Ejecución

Tipos de permisos en Windows

En Windows, los **permisos de archivos** controlan qué acciones puede hacer un usuario sobre un archivo o carpeta:

Permiso	Significado
Lectura	Ver el contenido del archivo
Escritura	Modificar o eliminar el archivo
Ejecución	Ejecutar el archivo si es un programa

Cambiar permisos manualmente (interfaz gráfica)

- Haz clic derecho sobre el archivo o carpeta.
- Selecciona `Propiedades`.
- Ve a la pestaña `Seguridad`.
- Haz clic en `Editar` para modificar permisos.
- Marca o desmarca permisos para cada usuario o grupo.

Cambiar permisos por terminal (PowerShell)

- Puedes usar `icacls` para ver o cambiar permisos.

```
icacls "C:\ruta\archivo.txt"
```

- Dar permisos de escritura al usuario actual:

```
icacls "C:\ruta\archivo.txt" /grant %username%:W
```

- Dar permisos de lectura y ejecución:

```
icacls "C:\ruta\archivo.bat" /grant %username%:RX
```

Usar permisos desde Java

En Java, puedes verificar o cambiar permisos con métodos de la clase `File`:

```
import java.io.File;

public class Permisos {
    public static void main(String[] args) {
        File archivo = new File("C:\\ruta\\archivo.txt");

        System.out.println("Se puede leer: " + archivo.canRead());
        System.out.println("Se puede escribir: " + archivo.canWrite());
        System.out.println("Se puede ejecutar: " + archivo.canExecute());

        archivo.setReadable(false);
        archivo.setWritable(true);
        archivo.setExecutable(false);
    }
}
```

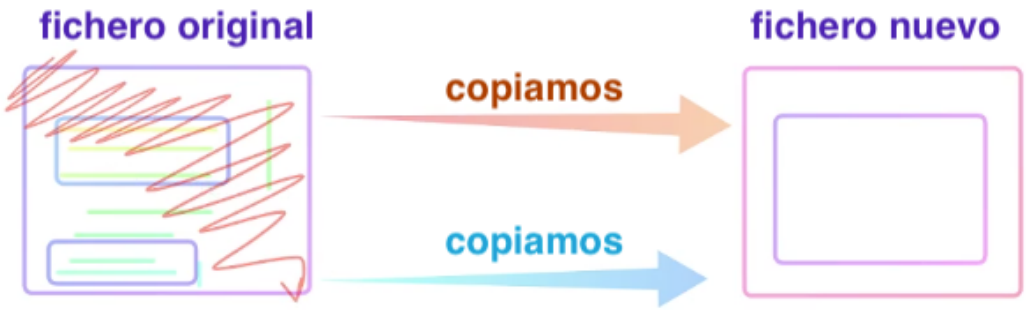
Los métodos `setReadable`, etc., solo afectan a archivos creados por el programa, y no siempre tienen efecto en Windows dependiendo del sistema de archivos.

Recomendaciones:

- Usa la interfaz gráfica para usuarios finales.
- Usa `icacls` en scripts o administración avanzada.
- En Java, verifica permisos antes de acceder a archivos para evitar errores.

Operaciones con ficheros

- Creación: Crea un nuevo fichero con un nombre, extensión y ruta determinados.
- Apertura: El programa abre el fichero y se prepara para leerlo o escribirlo. Suele "reservar" el fichero para sí.
    - Lectura: Lee el fichero o una de sus partes. Se debe disponer del permiso de lectura sobre el fichero. Es una operación NO bloqueante (varios programas pueden leer a la vez el mismo fichero).
    - Escritura: Permite escribir en el fichero, ya sea añadiendo datos o sobrescribiendo los ya existentes. Se debe disponer del permiso de escritura sobre el fichero. Es una operación bloqueante (un solo programa va a poder escribir al mismo tiempo sobre el mismo fichero). La escritura de datos con lleva consigo tres operaciones:
        - Altas: Añadir info al fichero.
        - Modificaciones: Modificar info del fichero, es decir, cambiar partes del fichero.
        - Bajas: Dependiendo del tipo de acceso se realizará de una forma o de otra (ver "*Formas de Acceso a un Fichero*"):



Copiamos solo los registros que queremos mantener al nuevo fichero y posteriormente eliminamos el fichero original

- ✓ Bajas Secuenciales: Se crea un nuevo archivo sin los datos que se desean.
- ✓ Bajas Aleatorios: Se desactiva el registro para posteriormente cuando se haga una modificación sobreescribirlo.
    - Valor 1: el registro está activo y no se puede sobreescribir.
    - Valor 0: el registro NO está activo y se puede sobreescribir.

NOTA: Esta segunda opción es más optima porque no hay que copiar todo el fichero de un sitio a otro, pero en esta se gastan más bits. Nos deberíamos de preguntar que es mejor perder mucho tiempo, perder un poco de memoria? Actualmente como la memoria no es problema… mejor perder un poco de memoria.

- Cierre: Indica que se ha finalizado con las operaciones sobre el fichero. Libera el fichero para que el resto de programas o el SO pueda hacer uso del mismo.

NOTA: Siempre que abramos un fichero, sockets para comunicaciones de Internet, a controladores de BBDD a cualquier cosa que os podáis imaginar que abráis desde un programa tendréis que cerrar para dejarlo disponible.

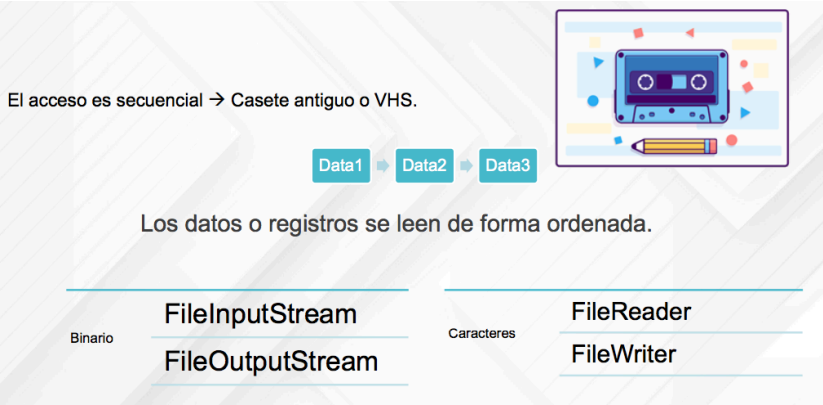
En Java se permiten hacer todas las operaciones con ficheros vistas anteriormente.

Antes de meternos de lleno con cada una de las operaciones, tenemos que hablar de las formas de acceso a un fichero.

Formas de acceso a un fichero

En este apartado, explicaremos las diferentes maneras de acceder a la información de un fichero. Podemos diferenciar dos tipos de acceso: el acceso secuencial y el acceso aleatorio.

Secuencial



El acceso se produce desde el primer registro y se va avanzando registro a registro para leer la información. Si deseamos acceder al registro 24 debemos leer los 23 anteriores.

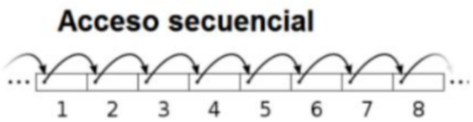
Cómo podemos observar son los menos óptimos ya que es obligatorio recorrer de forma constante el fichero desde el inicio.

Un archivo con acceso secuencial es un archivo donde se guardan los registros en orden, con base en el campo clave de origen. Para leer los datos de un fichero secuencial, se debe acceder a los datos uno después de otro y, una vez consultados, no se podrá acceder a ellos si no se sigue el orden.



Un ejemplo puede ser un VHS, (película antigua) donde para ir a un fragmento de la misma debíamos avanzar o rebobinar sobre la cinta electromagnética para leer su información

Aquí tenemos la simplificación a modo de esquema para aclarar el concepto.



Más adelante, veremos ejemplos prácticos de lectura y escritura de este tipo de acceso de datos.

Aleatorio



Se puede acceder directamente a un registro sin haber recorrido los anteriores. Casi todos los sistemas actuales utilizan este formato ya que es mucho más rápido que el secuencial.

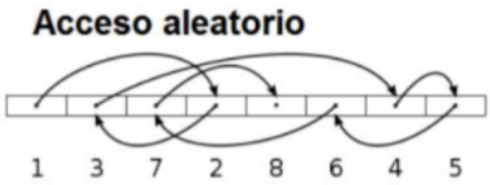
El acceso aleatorio a archivos es un tipo de acceso a datos que permite al programa Java acceder a los datos sin un orden, es decir, a cualquier posición en que se encuentren los datos.



Un ejemplo es un DVD donde podemos avanzar y retroceder a nuestro antojo desde cualquier punto.

Pasaría lo mismo con un vídeo online, es un fichero al que podremos acceder a cualquiera de sus segundos en cualquier momento sin tener que ver todo el vídeo entero.

A diferencia del acceso secuencial, con el acceso aleatorio no es necesario empezar desde la primera línea del fichero. Se asemeja a los arrays de bytes, ya que podemos acceder a cualquier parte de los datos con un puntero de fichero, es decir, indicando al método de acceso en qué posición queremos empezar a tratar los datos. En la siguiente ilustración, podremos apreciar más fácilmente a qué nos referimos con acceso aleatorio de los datos.



Como podemos apreciar, el acceso a los datos es totalmente aleatorio y sin ningún orden, y este tipo de acceso es realmente útil en aquellas aplicaciones de baja latencia que necesitan persistencia. Este tipo de acceso a datos es especialmente útil para situaciones en que el programa ha sufrido un error y es necesario acceder otra vez de manera aleatoria a los datos.

Ventajas e inconvenientes de las distintas formas de acceso

Las ventajas del acceso de datos secuencial frente al aleatorio es que este tipo de estructura se puede usar para persistir datos de manera más simple.

Por otro lado, las desventajas son evidentes: no es una manera de acceder a los datos de manera eficiente. Nos obliga a acceder a los datos en orden, aunque el resto de datos no nos interesen.

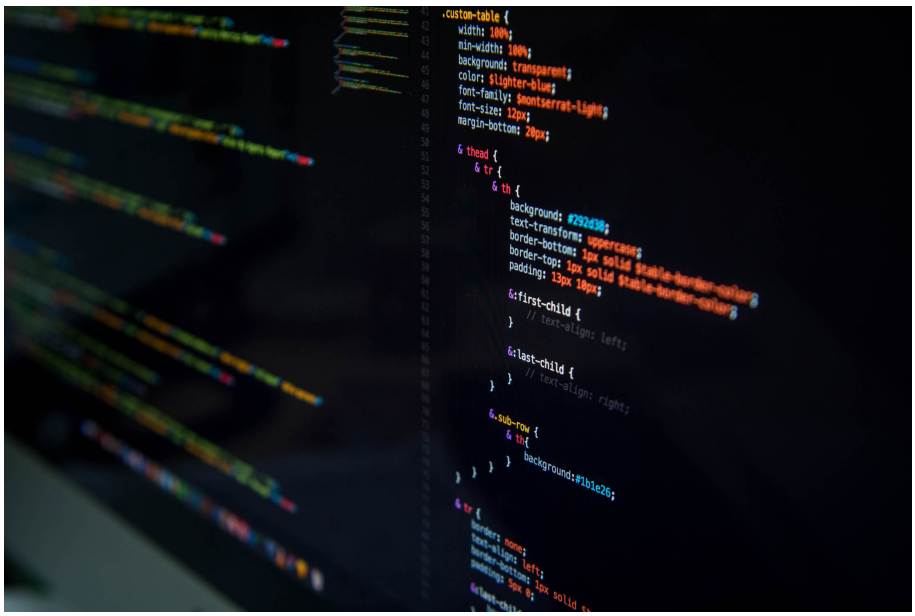
En cambio, el acceso de datos aleatorio proporciona mucho más control y eficiencia a la hora de tratar la información. Por un lado, podemos elegir el punto exacto de lectura de datos, permitiendo ser mucho más rápido y ahorrar consumo de recursos en la aplicación. Como desventaja podemos destacar que puede ser un método de acceso un poco más lento en comparación con el secuencial.

Clases Java de acceso a un fichero

- Secuencial :
  - ✓ FileInputStream / FileOutputStream
  - ✓ FileReader / FileWriter
- Aleatorio o directo :
  - ✓ RandomAccessFile

Clases asociadas a operaciones de gestión de ficheros y directorios

Cuando se programa con Java, podemos realizar tareas básicas de gestión de ficheros desde crear hasta leer, borrar, copiar o mover de sitio carpetas o archivos.



Para tratar los diferentes archivos, Java tiene diferentes paquetes que nos pueden ayudar a realizar las operaciones básicas con cualquier tipo de fichero, tal y como veremos a continuación.

Los diferentes objetos que veremos en esta sección pertenecen a diferentes paquetes de Java, algunos de la librería propia de Java (incluidas en el JDK de Java), pero también hay otros que son necesario importación.

Librerías específicas utilizadas en la gestión de ficheros

En Java, un paquete ó librería se puede entender como un conjunto de clases que facilitan operaciones y tareas ofreciendo al programador funcionalidades ya implementada y listas para ser usadas a través de una Interfaz de Programación de Aplicaciones, API (Application Programming Interface).

Ventaja de los paquetes: Usamos paquetes para evitar conflictos de nombres y para escribir un código más fácil de mantener. Los paquetes se dividen en dos categorías:

- Paquetes integrados (paquetes de la API de Java)
- Paquetes definidos por el usuario (crea tus propios paquetes)

Esto significa que se puede importar una sola clase (junto con sus métodos y atributos) o un paquete completo que contenga todas las clases que pertenecen al paquete especificado.

Para utilizar una clase o un paquete de la biblioteca, se debe utilizar la import palabra clave.

Ejemplo de importación de unas librerías:

```
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
```

A continuación hablaremos de tres librerías utilizadas en la gestión de las operaciones de entrada y salida con ficheros:

- Java.io (Java Input Output)
- Java.nio (Java Non-blocking Input Output)

1. java.io (Java Input Output)



El paquete más utilizado en el lenguaje Java es el paquete ó librería java.io. Se encarga de gestionar las operaciones de entrada y salida de nuestro programa .

Es muy usada, ya que las operaciones System.out y System.err la utilizan para poder mostrar los mensajes.

A continuación se mencionan algunas de las clases más relevantes de esta paquete, java.io :

- java.io.File -> Es la clase File de gestión de ficheros.
- java.io.IOException - > Es un tipo de excepción generalizada que es lanzada cuando algún tipo de operación I/O ha fallado. Esta excepción debe de ser tratada ya sea manejándola con un bloque de código try-catch en el mismo momento que se realice la operación o dejando que se propague hacia arriba en la pila de llamadas utilizando throws IOException.

A continuación haremos hincapié en la clase File, también veremos cómo filtrar ficheros con esta librería java.io, es decir, obtener aquellos ficheros con una característica determinada, como puede ser que tengan la extensión .java, o la que nos interese, y más adelante, también veremos como crear y eliminar ficheros y directorios con esta clase.

❖ Clase File (Java.io.File)

La clase File manipula los ficheros y directorios en Java.



La clase File nos permite realizar operaciones con ficheros y directorios, hay que resaltar que NO nos permite ni leer ni escribir pero sí la necesitamos para poder realizar estas operaciones.

La clase File trabaja con streams, un flujo de datos (en formato de bytes).

Un stream es una librería de java.io que se utiliza para gestionar flujos de datos, ya sea en ficheros, strings o dispositivos. Se encarga de tratar de manera ordenada una secuencia de datos con un origen y un destino.

Los datos que se leen desde este paquete no se guardan en ningún sitio, es decir, no se guardan en caché. Si fuera necesario recorrerlos otra vez, un *stream* no sería útil, sino que necesitaríamos cambiar a un búffer, pero ampliaremos información más adelante en el tema.

La clase File permite realizar un gran número de operaciones:

- Apertura y cierre de un fichero.
- Comprobar si el fichero existe.
- Crear un fichero.
- Borrar un fichero.
- Obtener nombre, rutas (absolutas y relativas) y extensión del fichero.
- Decir si el fichero es un fichero o un directorio.
- Obtener el tamaño en bytes del fichero.
- Consultar y cambiar los permisos del fichero.
- Si es un directorio, obtener la lista de ficheros que contiene el mismo.
- Crear nuevos directorios.
- Renombrar fichero…

Consulta todas la operaciones que se pueden realizar en https://www.w3api.com/Java/File/, https://docs.oracle.com/javase/7/docs/api/java/io/File.html

El resto de clases que manipulan ficheros parten de la existencia de una clase File, por lo que es la base de cualquier operación de manipulación de ficheros.

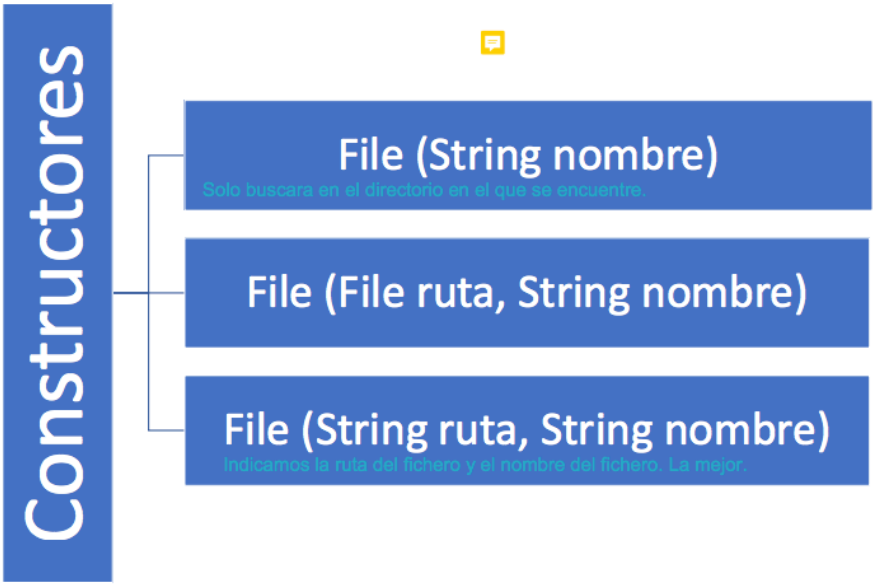
❖ Manejo de ficheros en Java con la clase File

Para crear un nuevo objeto, debemos declarar una nueva instancia del objeto *File*, y le pasaremos la ruta, el nombre del fichero y la extensión en un *string*.

Un objeto de la clase File representa un fichero, apunta mediante su ruta a un fichero o directorio del sistema de archivos o también se puede crear un nuevo fichero.

✓ NO representa el contenido de ningún fichero.

Dependiendo de la plataforma en la que nos encontremos (Linux, Mac o Windows) cambiarán las rutas de los directorios de los archivos, además en los sistemas Linux y Mac habrá que tener en cuenta que en las rutas se distinguen mayúsculas de minúsculas, cosa que no ocurre en los sistemas Windows.



La clase File, puede utilizar cualquiera de los siguientes constructores:

- **File(String path)**
  - ✓ Linux:

```
File varFichero = new File (“/directorio/subdirectorio/fichero1.txt”);
```

  - ✓ Windows:

```
File varFichero = new File(“c:\\directorio\\subdirectorio\\fichero1.txt”);
```

- **File(String path, String name)**
  - ✓ Linux:

```
File varFichero= new File(“/directorio/subdirectorio”,”fichero1.txt”);
```

- **File(File path, String name)**
  - ✓ Linux:

```
File varFichero= new File(new File (“/directorio/subdirectorio”),”fichero1.txt”);
```

Dependiendo de nuestras necesidades podemos utilizar uno u otro.

En Java el separador de archivos tanto para Windows como para Linux es el símbolo /, aunque las rutas se escriben en: Linux / y en Windows \\

Al crear el constructor, la variable varFichero es un objeto con los datos del fichero que se encuentra en la ruta pasada por parámetro, si es que existe:

- La ruta puede ser absoluta o relativa.
- Para cerrar un fichero, se usa la siguiente sentencia:

```
varFichero.close();
```

Con esta sentencia se cierra el fichero y la variable varFichero pasará a ser nula (null).

❖ Constantes de la clase File

En la clase File tenemos un par de constantes muy útiles, que nos servirán si usamos nuestra aplicación en varios sistemas operativos. Los sistemas operativos manejan rutas y listas de rutas de manera distinta:

Concepto	**Windows**	**Unix/Linux/macOS**
Separador de directorios	\	/
Separador de listas de rutas	;	:

Utilizar estas constantes nos evitará duplicar código a la hora de trabajar con ficheros dependiendo si es un sistema Windows o Unix.

Los métodos son:

- separator: Este es el separador de directorios en rutas de archivos.
  Separador de directorio (File.separator) se usa dentro de una ruta para unir carpetas y nombres de archivos. P.e:

```
String ruta = "C:" + File.separator + "carpeta" + File.separator + "archivo.txt";
// Windows -> "C:\carpeta\archivo.txt"
// Linux -> "C/carpeta/archivo.txt"
```

- pathSeparator: Este es el separador para **listas de rutas** (como en variables de entorno PATH).

```
 String variasRutas = "C:\\Windows" + File.pathSeparator + "C:\\Program Files";
// Windows -> "C:\Windows;C:\Program Files"
// Linux -> "/usr/bin:/usr/local/bin"
```

Ejemplo:

```
String carpeta = "docs";
String archivo = "datos.txt";

// Forma portátil
String ruta = carpeta + File.separator + archivo;
File f = new File(ruta);
System.out.println(f.getAbsolutePath());
```

Por qué es importante? Si escribes rutas hardcodeadas como "C:\\carpeta\\archivo.txt":
Funcionará en Windows, pero no en Linux o macOS.

❖ Métodos de la clase File

A diferencia de otras clases que manejan ficheros, en la clase File no es necesario controlar excepciones en la mayoría de los métodos. Si hay alguna excepción a manejar lo indicaremos.
Estos son los métodos mas conocidos:

Método	Descripción	Ejemplo	Dato devuelto
Exists ()	Indica si existe o no el fichero.		boolean
isDirectory ()	Indica si el objeto File es un directorio.		boolean
isFile ()	Indica si el objeto File es un fichero.		boolean
isHidden ()	Indica si el objeto File esta oculto.		boolean
getPath ()	Obtiene la ruta del fichero, el inicio es el punto de ejecución del programa	File ficheros = new File("ficheros"); System.out.println(ficheros.getPath());	String
getAbsolutePath ()	Devuelve una cadena con la ruta absoluta del fichero o directorio.	File ficheros = new File("ficheros"); System.out.println(ficheros. getAbsolutePath());	String
canRead ()	Indica si se puede leer.	File ficheros = new File("ficheros"); System.out.println(ficheros. canRead());	boolean
canWrite ()	Indica si se puede escribir.	File ficheros = new File("ficheros"); System.out.println(ficheros. canWrite());	boolean
canExecute ()	Indica si se puede ejecutar.		boolean
getName ()	Devuelve una cadena con el nombre del fichero o directorio.		String
getParent ()	Devuelve una cadena con el directorio padre.	File ficheros = new File("ficheros"); System.out.println(ficheros. getParent());	String
listFiles ()	Devuelve un array de File con los directorios hijos. Solo funciona con directorios.	File ficheros = new File("ficheros"); ficheros.listFiles();	Array de File
list ()	Devuelve un array de String con los directorios hijos. Solo funciona con directorios.	File ficheros = new File("ficheros"); ficheros.list();	Array de String
mkdir ()	Permite crear el directorio en la ruta indicada. Solo se creara si no existe.	File ficheros = new File("ficheros"); System.out.println(ficheros. mkdir());	boolean
mkdirs ()	Permite crear el directorio en la ruta indicada, también crea los directorios intermedios. Solo se creara si no existe.		boolean
createNewFile ()	Permite crear el fichero en la ruta indicada. Solo se creara si no existe. Debemos controlar la excepcion con IOException.	File ficheros = new File("ficheros"); System.out.println(ficheros. createNewFile());	Boolean
delete ()	Elimina un fichero o directorio, si es un directorio debe estar vacío	File ficheros = new File("ficheros"); System.out.println(ficheros. delete());	Boolean
renameTo ()	Renombra un fichero o directorio	File ficheros = new File("ficheros"); System.out.println(ficheros.renameTo(ne w File("eliminados")));	Boolean

Indica si la afirmación es verdadera o falsa:

Podemos establecer la fecha de modificación de un archivo mediante el método renameTo().
a. Verdadero
b. Falso

Cuando trabajamos con ficheros en Java, no es necesario capturar las excepciones, el sistema se ocupa automáticamente de ellas.
a. Verdadero
b. Falso

❖ Interface FilenameFilter: Creación de un filtro

Un filtro es un objeto de una clase que implementa el interface *FilenameFilter*, y tiene que redefinir la única función del interface denominada *accept*. Esta función devuelve un dato de tipo boolean. En este caso, la hemos definido de forma que si el nombre del archivo termina con una determinada extensión devuelve true en caso contrario devuelve false. La función *endsWith* de la clase *String* realiza esta tarea tal como se ve en la porción de código que viene a continuación. La extensión se le pasa al constructor de la clase *Filtro* para inicializar el miembro dato *extension*.

```
import java.io.File;
import java.io.FilenameFilter;

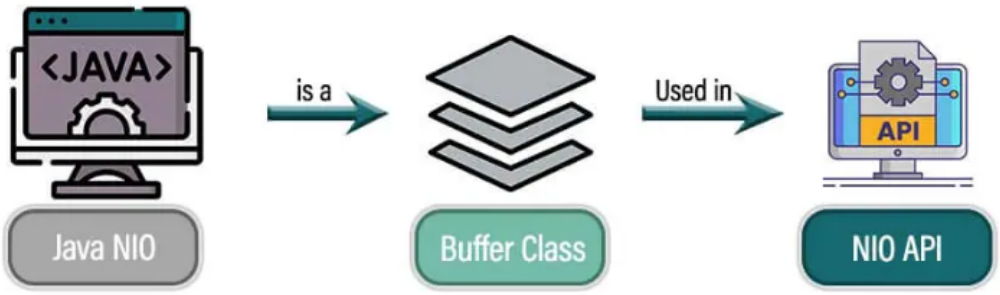
public class Filtrar implements FilenameFilter {
   String extension;
   // Constructor
   Filtrar(String extension){
           this.extension = extension;
    }
    public boolean accept(File dir, String name){
           return name.endsWith(extension);
    }
   public static void main(String[] args) {
   try {
     // Obtendremos el listado de los archivos de ese directorio
     File fichero=new File("c:\\datos\\.");
```

```
     String[] listadeArchivos = fichero.list();
      // Filtraremos por los de extension .java
      listadeArchivos = fichero.list(new Filtrar(".java"));
      // Comprobamos el número de archivos en el listado
      int numarchivos = listadeArchivos.length ;
      // Si no hay ninguno lo avisamos por consola
      if (numarchivos < 1)
          System.out.println("No hay archivos que listar");
          // Y si hay, escribimos su nombre por consola.
          else
       {
              for(int conta = 0; conta < listadeArchivos.length; conta++)
              System.out.println(listadeArchivos[conta]);
       }
} catch (Exception ex) {
       System.out.println("Error al buscar en la ruta indicada"); }
}
```

Para obtener la lista de archivos con extensión .java en el directorio actual, creamos un objeto de la clase *Filtrar* y se lo pasamos a la función *list* miembro de la clase *File*.

```
listadeArchivos = fichero.list(new Filtrar(".java"));
for ( int i=0; i<listaArchivos.length; i++ ) {
         System.out.println(listaArchivos[i]);
}
```

2. java.nio (Java Non-blocking Input Output)



Es una librería importante que encontraremos en la versión de Java 8. Es otra alternativa para la creación de ficheros, una de las más recomendadas. Tiene un gran repertorio de métodos para la creación, copia, borrado, escritura y lectura de datos.

Esta segunda librería supone una mejora en la forma en la que se realizan las operaciones. Ambas pueden ser utilizadas, pero java.nio corrigió muchas de las deficiencias de java.io.

Entrando en detalle de este paquete, nos permite un enfoque diferente a la hora de gestionar los datos: java.nio.file permite el almacenaje de la información del fichero en un búfer.

Un búfer es un bloque de memoria que permite almacenar temporalmente los datos y recorrerlos tantas veces como se desee para tratarlos.

Tal y como hemos comentado en la definición de búfer, los datos que se guardan en él se pueden volver a leer según se necesite. Nos proporciona un poco más de flexibilidad a la hora de tratar con un fichero.

A continuación se mencionan algunas de las clases más relevantes de esta paquete:

- java.nio.file.Path -> Sirve para manejar las rutas de los ficheros.
- java.nio.file.Files -> Sirve para controlar las operaciones básicas de los ficheros.
- java.nio.file.FileSystem -> Sirve para obtener referencias al sistema de archivos.

Esta librería consta con métodos para la manipulación de ficheros que nos serán realmente útiles. Las operaciones principales a realizar con archivos y directorios son:

- Verificación de existencia y accesibilidad
- Borrar un archivo o directorio
- Copiar un archivo o directorio
- Mover un archivo o directorio

❖ Métodos de la clase PATH

Esta clase tiene métodos no estáticos y métodos estáticos (recordad "método estático" se puede acceder a él sin crear un objeto de la clase) para el manejo de ficheros, los métodos de la clase Files trabajan sobre objetos Path.

Métodos Estáticos:

- Of(String path)
- Of(String path, String…more)

Método No-Estáticos:

- getParent()
- normalize()
- toAbsolutePath()
- toFile()
- toString()
- toUrl()

❖ Métodos de la clase Files

Método	Descripción	Dato devuelto
createDirectory(path)	Crea un directorio en la ruta indicada.	path
createFile(path)	Crea un fichero en la ruta indicada.	path
copy(path origen, path destino)	Crea una copia de un fichero origen a una ruta destino que le indiquemos.	path
createTempDirectory(String prefijo)	Crea una carpeta temporal en la aplicación con el nombre que le pasemos como string al llamar al método.	Path
createTempFile(String prefijo, String sufijo)	Este método se encarga de crear un fichero temporal en la carpeta temporal del programa, usando el prefijo y el sufijo que le hemos indicado por parámetro.	path
delete(path)	Borra un fichero de la ruta indicada.	void
deleteIfExists(path)	Borra un fichero siempre que exista en la ruta indicada.	boolean
exists(path)	Comprueba si el fichero que le indicamos existe en la ruta indicada.	boolean
isDirectory(path)	Comprueba si ese fichero indicado en la ruta es una carpeta o no.	boolean
size(path)	Devuelve el tamaño del fichero que le indiquemos en la ruta.	long
walkFileTree(path)	Este método se encarga de recorrer el árbol de directorios de una ruta recursivamente.	Path
readAllLines(path)		
readAllBytes(path)		
write(path, byte[])		
writeString(path,CharSequence)		

Ejemplo listado de archivos

```
Path path = Path.of("ficheros");
try {
          Stream<Path> files = Files.list(path);
          For(Path path1: files.collect(Collectors.toList()))
          {
                    System.out.println(path1); }
} catch (IOException e) {
          System.err.println("Error al leer: "+path.getFileName()); e.printStackTrace();
}
```

Ejemplo escribir bytes

```
Path path = Path.of("ficheros/nio.txt");
byte[] a = { 20, 10, 30, 5 };
System.out.println("Byte[] inicial");
for (byte item: a){
          System.out.println(item);
}
try {
          Files.write(path, a);
} catch (IOException ex) {
          System.err.println("Error al escribir los bytes");
}
```

Ejemplo leer bytes

```
Path path = Path.of("ficheros/nio.txt");
try {
          byte[] b = Files.readAllBytes(path);
          System.out.println("Byte[] recuperado");
          for (byte item: b){
                    System.out.println(item); }
}catch (IOException ex){
          System.err.println("Error al leer los bytes");
}
```

Ejemplo escribir caracteres

```
Path path = Path.of("ficheros/nio.txt");
try {
          Files.writeString(path, "Esto es un texto de prueba");
} catch (IOException ex) {
          System.err.println("Error al escribir en el fichero: " + path.getFileName());
}
```

Ejemplo copia fichero

```
Path path = Path.of("ficheros/nio.txt");
Path copy = Path.of("ficheros/nio_copia.txt");
try {
          Files.copy(path, copy);
}catch (IOException ex){
          System.err.println("No se ha podido copiar el fichero");
}
```

Ejemplo mover fichero

```
Path path = Path.of("ficheros/nio.txt");
Path move = Path.of("ficheros/tema1/nio.txt");
try {
          Files.move(path, move);
} catch (IOException ex) {
          System.err.println("No se ha podido mover el fichero");
}
```

Operaciones básicas de ficheros

Existen 4 operaciones sobre los ficheros esenciales para la gestión de los mismos:

- Creación
- Eliminación
- Copia
- Movimiento

Por desgracia no todas ellas se pueden realizar con el API básico de Java.io.

Existe otra operación que no está incluida en este bloque pero haremos hincapié en ella, el renombramiento de ficheros:

- rename

```
import java.io.File;


public class fileRename {
    public static void main (String[] args) {
        System.out.println("Iniciando programa...");
        File file = new File("ficheros/fileIO_1.txt");
        File rename = new File("ficheros/fileIO_2.txt");
        if (file.renameTo(rename)){
            System.out.println("Fichero renombrado con éxito");
        }else{
            System.out.println("Fichero NO renombrado");
        }
        System.out.println("Finalizando programa...");
    }
}
```

A continuación trataremos las distintas operaciones que se pueden realizar con las distintas librerías.

Creación de un fichero

❖ Utilizando java.io.file

A continuación, veremos cómo crear un archivo *File* pasándole como parámetro al constructor la ruta, el nombre y la extensión del fichero que queremos crear. El constructor, lo que realmente necesita es un *string*, que será la ruta compuesta.

```
import java.io.File;
import java.io.IOException;;

public class fileCreator {
    public static void main (String[] args) {
        System.out.println("Iniciando programa...");
        File directorio= new File("ficheros");
        if (!directorio.mkdir()){
            System.err.println("No se ha podido crear el directorio, puede que exista");
        }
        File ficheroDatos = new File(directorio,"fichero1.txt");
        try{
            if (ficheroDatos.createNewFile()){
                System.out.println("Creado el fichero "+ ficheroDatos);
            }
            else {
                System.out.println("No se ha creado el fichero, puede que exista.");
            }
        }catch(IOException ex){
            System.err.println("No se ha podido crear el fichero");
            System.err.println(ex.getMessage());
            System.exit(-1);
        }
        System.out.println("Finalizando programa...");
    }
}
```

❖ Utilizando java.nio.file.Files

En este caso no utilizamos un *string* para definir la ruta del fichero que queremos crear, sino que utilizamos el objeto *Path*. Este es un objeto que se utiliza para localizar un fichero dentro de un sistema de archivos y se encargará de representar una ruta del fichero de nuestro sistema. La librería *Files* necesitará que se le pase este objeto para crear el fichero. Para crear el fichero, llamaremos al método *createFile()*, el cual devuelve un *path* con la ruta del fichero creado.

Al definir el *path*, tenemos que asegurarnos de que la ruta absoluta que utilizamos sea correcta, si no es así, se producirá un error cuando el programa ejecute esta parte del código porque no encontrará la ruta de carpetas indicada.

```
public class fileCreatorNIO {
    public static void main (String[] args) {
        System.out.println("Iniciando programa...");
        Path directorio = Paths.get("./ficheros/fileNIO_1.txt");
        if (Files.notExists(directorio)) { //Comprueba que no exista el fichero
            try{
                directorio = Files.createFile(directorio); //Crea el fichero
                System.out.println("Creado el fichero "+ directorio);
            }catch(IOException ex){
                System.err.println("No se ha creado el fichero");
                System.exit(-1);
            }
        }
        System.out.println("Finalizando programa...");
    }
}
```

Creación de un directorio

❖ Utilizando java.io.file

Para la creación de directorios se usa la misma clase *File* que hemos comentado anteriormente, pero el método para crearlo es algo distinto a la creación de ficheros. Si nos fijamos en el ejemplo:

```
import java.io.File;


public class dirCreatorIO {
        public static void main (String[] args) {
            System.out.println("Iniciando programa...");
            File directorio= new File("directorioIO1");
            if (directorio.mkdir()){ //Crea el directorio
                System.out.println("Creado el directorio"+ directorio);
            } else {
                System.out.println("No se ha podido crear el directorio");
            }
            System.out.println("Finalizando programa...");
        }
}
```

Como podemos ver, para crear un directorio el objeto *File* tiene un método que permite la creación de directorios: *mkdir()*. En el caso de que se realice con éxito, el método devuelve un booleano *true*.

Algunos de los problemas que nos podemos encontrar con este método son que, si no existe la carpeta indicada, o si la ruta absoluta que utilizamos para crear nuestro directorio no está bien, dará un error en la ejecución de nuestro programa. Debemos tener en cuenta esta posibilidad y controlar los errores.

❖ Utilizando java.nio.file.Files

El objeto java.nio.file.Files también nos ofrece posibilidad de creación de directorios a través del método *createDirectories()*, al cual le tendremos que pasar la ruta, como en otros objetos. Aquí podemos apreciar un ejemplo de cómo usar este objeto con el método de creación de carpetas.

```
public class dirCreatorNIO {
    public static void main (String[] args) {
        System.out.println("Iniciando programa...");
        String nombre = ".pepe/directorioNIO";
        Path ruta = Paths.get(nombre);
        try{
            Files.createDirectories(ruta);
            System.out.println("Creado el directorio"+ ruta);
        }catch(IOException ex){
            System.err.println("Se ha producido un error");
            System.exit(-1);
        }
        System.out.println("Finalizando programa...");
    }
}
```

Este tipo de tareas son especialmente útiles para diferenciar carpetas y tener organización dentro de un ordenador o un SFTP para gestionar archivos que se van generando.

Un SFTP (protocolo de transferencia de archivos) es un programa estandarizado para transferir archivos entre ordenadores de cualquier sistema operativo.

Por ejemplo, un usuario sube en una página web y que queremos transferir a un servidor para que queden almacenados. Una de las posibles utilidades sería crear carpetas por día, y crear nuevos archivos en esa carpeta.

Borrado de un fichero

❖ Utilizando java.io.file

En el objeto *File*, tenemos dos métodos:

- delete(): borrará el fichero que le indiquemos según la ruta o lanzará un error si no lo encuentra.

```
import java.io.File;


public class fileDeleteIO {
    public static void main (String[] args) {
        System.out.println("Iniciando programa...");
        File ficheroDatos = new File("./ficheros/fileIO_1.txt");
        if(ficheroDatos.delete()) {
            System.out.println("Hemos borrado un fichero, "+ ficheroDatos);
        }else{
            System.out.println("No se puede borrar el fichero, " + ficheroDatos);
        }
        System.out.println("Finalizando programa...");
    }
}
```

No es necesario crear un nuevo objeto para borrar el fichero en cuestión, solo necesitamos indicarle la ruta y el nombre del fichero, aunque también existe la posibilidad de borrar un fichero ya creado con el mismo método.

- deleteOnExit(): este método, en cambio, borrará el fichero o el directorio que le indiquemos por ruta absoluta cuando la máquina virtual finalice.

```
File archivo = new File("src/main/resources/crearFicheros fichero4.txt");
archivo.deleteOnExit()) {
          System.out.println("Hemos borrado un fichero");
}
```

❖ Utilizando java.nio.file.Files

La librería java.nio.Files también nos ofrece la posibilidad de borrar ficheros, aunque el procedimiento es algo distinto. También cuenta con un método *delete()* para borrar los ficheros. Para realizar un borrado, se debe realizar de este modo:

```
//Primero crearemos una carpeta, en este caso se crea porque no existe y sino nos dara error.
String nombrePath = "src/main/resources/crearFicheros/directorio";
File ficheroCarpeta = new File(nombrePath); ficheroCarpeta.mkdir();
//Indicamos la ruta con el directorio y el fichero que queremos crear
String nombrefichero = "src/main/resources/crearFicheros /ejercicio/fichero.txt";
File ejemplo = new File(nombrefichero);
ejemplo.createNewFile();
//Cogemos la ruta y lo borramos. Si no existe dara error
Path path = Paths.get(nombrefichero);
try {
          Files.delete(path);
}catch (IOException e) {
          System.out.println("No existe la carpeta" + ficheroCarpeta);
}
```

Para el borrado de directorios, se puede usar los mismos métodos que hemos indicado anteriormente. En el caso de directorios, antes de borrar se debe comprobar que esté vacío, si no, no va a borrarse y se lanzara una excepción *NoSuchFileException*. Hablaremos de las excepciones más adelante.

El objeto java.nio.files.File también ofrece posibilidad de borrado de carpetas.

```
String nombrePath = "src/main/resources/crearFicheros/directorio";
Path carpeta = Paths.get(nombrePath);
Files.delete(carpeta);
```

Copia

Otra de las acciones que podemos realizar con los ficheros o directorios es el copiado. Este método nos será útil para copiar ficheros de un origen a un destino, y tratarlos sin modificar los datos de origen.

Tal y como hemos mostrado anteriormente, veremos ejemplos de los dos objetos Java más utilizados que disponen del método para copiar ficheros.

❖ Utilizando java.nio.file.File

Para empezar, la manera más fácil de copiar un archivo es con la API de java.nio. El objeto *Files* utiliza el método *copy()*, el cual necesita que se le pase por parámetro la ruta de origen y la ruta de destino, con un objeto *Path*. La copia fallará si el fichero destino ya existe, pero si utilizamos la opción *REPLACE_EXISTING* el fichero se sobrescribirá. Veremos en el ejemplo cómo usar esta opción.

Este método también se usa para los directorios, pero debemos tener en cuenta que solo se copiará la carpeta, no el contenido de esa carpeta.

```
Path origen = Paths.get("src/main/resources/copiarFicheros/ejemploCopia.txt");
Path ejemploOrigen = Files.createFile(origen);
Path destino = Paths.get("src/main/resources/copiarFicheros/ /destino");
Path ejemploDestino = Files.createFile(destino);
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
```

❖ Utilizando java.io.file

Por otro lado, el objeto *Files* que se encuentra en la API java.io ofrece también la opción de copiar ficheros. Actúa de una manera parecida, pero se implementa de modo distinto. Para realizar el copiado, es necesario crear búferes.

Aquí podemos ver un ejemplo práctico:

```
//Primero se crean los ficheros
File archivoOrigen = new File("src/main/resources/copiarrFicheros/origen.txt");
archivoOrigen.createNewFile();
File archivoDestino = new File("src/main/resources/copiarFicheros/destino.txt");
archivoDestino.createNewFile();
try {
        // Se lee el origen
```

```
            InputStream origen = new BufferedInputStream(new FileInputStream(archivoOrigen));
            // Fichero destino
            OutputStream destino = new BufferedOutputStream(new FileOutputStream(archivoDestino));
            byte[] buffer = new byte[1024];
            int lengthRead;
            while ((lengthRead = origen.read(buffer)) > 0) {
              // se escriben los datos de un fichero a otro
              destino.write(buffer, 0, lengthRead);
              // Se cierra el proceso
              destino.flush();
            }
            // Ejemplo de copiado de datos con la api java.nio
            Path orig = Paths.get("src/main/resources/copiarFicheros/ejemploCopia.txt");
            Path ejemploOrigen = Files.createFile(orig);
            Path dest = Paths.get("src/main/resources/copiarFicheros/destino");
            Path ejemploDestino = Files.createFile(dest);
            Files.copy(orig, dest, StandardCopyOption.REPLACE_EXISTING);
            System.out.println("se ha realizado la copia de ficheros");
} catch (Exception e) {
      e.getCause();
}
```

Tal y como vemos en el ejemplo, primero tendremos que crear un archivo origen y otro destino, con las rutas correspondientes. A continuación, necesitamos crear un *InputStream* con un nuevo *BufferedInputStream*, que se encargará de leer los datos del archivo de origen. Profundizaremos más adelante en estas clases, pero las clases *InputStream* y *BufferedInputStream* se caracterizan por leer archivos, y en este ejemplo nos serán útiles para coger el contenido del fichero origen y traspasarlo al fichero destino. En cambio, las clases *OutputStream*, *BufferedOutputStream* y *FileOutputStream* se encargan de escribir ficheros.

Para el objeto destino, debemos crear un *OutputStream* que contendrá un *BufferedOutputStream* para albergar el resultado de la copia. Con un bucle, recorreremos el fichero origen dato a dato y lo escribiremos al objeto destino. Al finalizar, se llamará al método *flush()* para terminar con el proceso. Este método se encargará de vaciar el *OutputStream* y se guardarán los archivos de salida en un búfer.

Para realizar la copia de los archivos es necesario muchas más líneas de código en comparación con el otro ejemplo, por eso recomendamos usar la API java.nio.

Movimiento

Java proporciona funciones para mover ficheros entre carpetas. Hay diferentes maneras de hacerlo, y veremos las posibilidades que tenemos según la API que queramos usar.

❖ Utilizando java.nio.file.Files

La API java.nio es la que ofrece una opción más rápida para mover un fichero. Usa el método *move()*, al cual le tendremos que pasar por parámetro el origen e indicarle un destino. Como se hacía en la copia, tenemos la posibilidad de indicarle por parámetro *REPLACE*_EXISTING. Este parámetro le indica al método que, si existe un dichero con ese nombre, lo deberá sobrescribir. Aquí tenemos un ejemplo de cómo se debe implementar:

```
Path destino = Files.move(Paths.get("c:/ejercicio/ejemplo.txt"), Paths.get("c:/ejercicio/destinoEjemplo.tx
t"), StandardCopyOption.REPLACE_EXISTING);
```

❖ Utilizando java.io.file

Otra manera de realizar esta acción es con el objeto *File* de la API java.io. Este objeto tiene una forma más rudimentaria de realizar el movimiento. Básicamente, se encarga de renombrar el fichero a uno nuevo y borrar el fichero de origen. Recomendamos por su rapidez usar el objeto *Files*.

```
File origenArchivo = new File("src/main/resources/crearFicheros/carpetaEjemplo/ficheroOrigen.txt");
origenArchivo.createNewFile();
if(origenArchivo.renameTo(new File(src/main/resources/crearFicheros/destino.txt"))){
          origenArchivo.delete();
          System.out.println("Se ha movido el fichero.");
} else {
          System.out.println("Se ha producido un error.");
}
```

EJERCICIOS BLOQUE 1

1. Crea un directorio llamado ”dam-accesodatos”
2. Crea un fichero llamado ejercicio1, dentro del directorio “dam-accesodatos”
3. Muestra por pantalla la longitud del fichero con nombre “ejercicio1”
4. Crea un fichero llamado ejercicio2 , dentro del directorio “dam-accesodatos”
5. Muestra todos los ficheros del directorio “dam-accesodatos”. *Pista: Puedes listarlos utilizando el método **list()** y meterlos en un array de ficheros para luego recorrerlos.*
6. Elimina el fichero llamado ejercicio1
7. Muestra de nuevo todos los ficheros del directorio “dam-accesodatos”
8. Elimina nuevamente el fichero llamado ejercicio1.
9. Borra directorio sin ficheros en su interior. *Pista: Puedes utilizar el método exists() y el delete()*
10. Borra directorio con ficheros en su interior. *Pista: Crea función recursiva borrarDirectorio*

A continuación se muestra como tiene que ser la estructura del programa:

```
//Aquí las librerías que necesites importar para trabajar con ellas

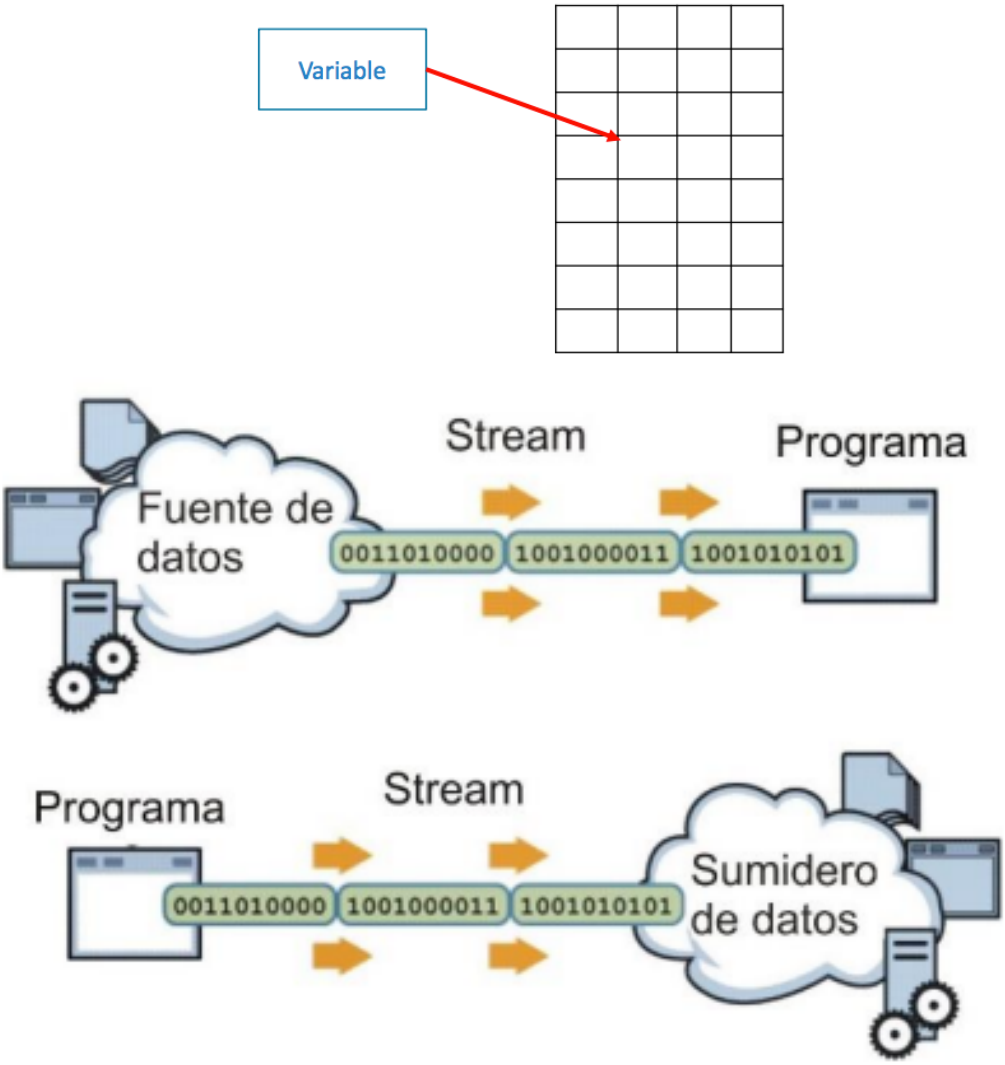

public class Bloque1 {
          public static void main(String[] args) {
          // 1. Crea un directorio llamado "dam-accesodatos"
          // 2. Crea un fichero llamado "ejercicio1" dentro del directorio "dam-accesodatos"
          // 3. Muestra por pantalla la longitud del fichero con nombre "ejercicio1"
          // 4. Crea un fichero llamado "ejercicio2" dentro del directorio "dam-accesodatos"
          // 5. Muestra todos los ficheros del directorio "dam-accesodatos"
          // 6. Elimina el fichero llamado "ejercicio1"
          // 7. Muestra todos los ficheros del directorio "dam-accesodatos"
          // 8. Elimina nuevamente el fichero llamado "ejercicio1"
          // 9. Borrar directorio sin ficheros en su interior
          // 10. Borrar directorio con ficheros en su interior


          }


          public static void borrarDirectorio(File directorio) {
          // Elimina los ficheros contenidos dentro del directorio de forma recursiva
          }
}
```

Gestión de flujos de datos: Streams – Acceso secuencial

Para poder realizar las operaciones de lectura y escritura de ficheros, Java establece lo que se conoce como un Stream de datos.
Crea una vía de comunicación entre programa y fichero que permite "moverse" por las distintas partes del fichero. Existe un puntero que apunta a las partes del fichero.



Los flujos de datos (streams en inglés) son una abstracción empleada en muchos lenguajes de programación, entre ellos Java, para representar cualquier fuente que produzca o consuma información. Su nombre (flujo) viene de que pueden considerarse como una cadena de datos continúa con una longitud que, posiblemente, es desconocida, al igual que la "longitud" de un flujo de agua que está corriendo.

Los flujos de datos o streams representan cualquier fuente que proporcione datos al programa, o cualquier sumidero que tome datos del programa.

Existen dos tipos de flujos de datos:

- los binarios o de bytes: la información que fluye está en formato binario. Se basan en enviar la información en bloques de 8 bits, 1 byte, también son conocidos como ficheros binarios. Estos proporcionan lo necesario para la gestión de entradas y salidas de bytes y su uso está orientado a la lectura y escritura de datos binarios. El tratamiento del flujo de bytes viene determinado por dos clases abstractas que son InputStream y OutputStream Estas dos clases definen los métodos que sus subclases tendrán implementados y, de entre todos, destacan read() y write() que leen y escriben bytes de datos respectivamente.

"Un fichero Word es un fichero BINARIO. Además de caracteres tiene tablas, imágenes… si se abre con el notepad no será ilegible"



"Un fichero PDF también es un fichero BINARIO."



- los de caracteres o de texto: la información que fluye está en formato texto. Se basan en enviar la información en bloques de 16 bits, 2 bytes, esto es debido a la codificación Unicode que usa 16bits para representar cada carácter(UTF-16), también conocidos como archivos de caracteres. De manera similar a los flujos de bytes, los flujos de caracteres están determinados por dos clases abstractas, en este caso: Reader y Writer. Dichas clases manejan flujos de caracteres Unicode. Y también de ellas derivan subclases concretas que implementan los métodos definidos en ellas siendo los más destacados los métodos read() y write() que leen y escriben caracteres de datos respectivamente.

Cada una de estas dos categorías puede volver a subdividirse en:

- flujos de datos de entrada: serían flujos que nos proporcionan datos, es decir, entradas de nuestro programa.
- flujos de datos de salida: flujos en los cuales nuestro programa escribe datos, es decir, salidas de nuestro programa.

Es muy importante determinar que tipo de stream necesitamos y hacer la programación adecuada. La elección nos hará utilizar unas clases u otras.

En este apartado vamos a ver las principales clases para representar flujos binarios y de texto, de entrada y de salida que proporciona Java.

Streams binarios o de bytes

Los streams de 8 bits, 1 byte, utilizan las clases:

- InputStream para entrada de datos: nos permite leer bytes de un array, de un String, de un fichero, etc.
- OutputStream para salida de datos: nos permite todo lo contrario hacer salida de datos a un array o un fichero.



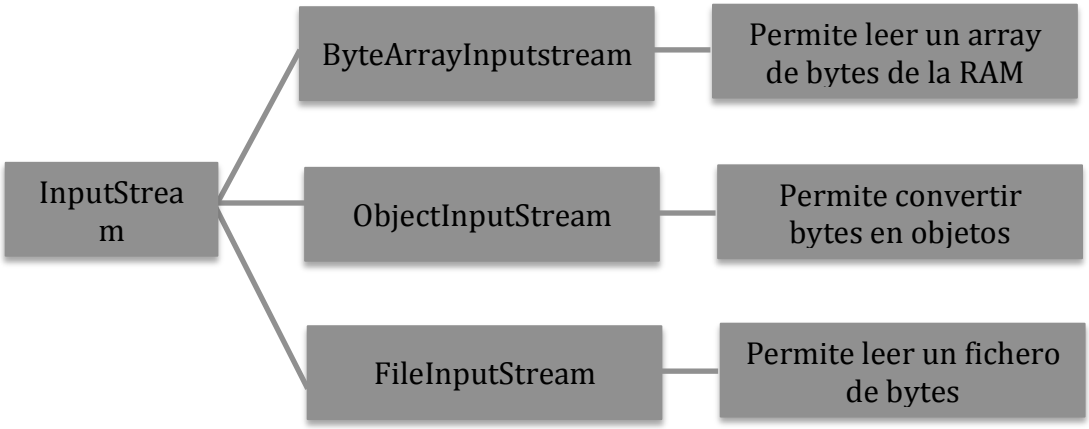
Las clases principales que heredan de OutputStream, para la escritura de ficheros binarios son:

- FileOutputStream: escribe bytes en un fichero. Si el archivo existe, cuando vayamos a escribir sobre él, se borrará. Por tanto, si queremos añadir los datos al final de éste, habrá que usar el constructor FileOutputStream(String filePath, boolean append), poniendo append a true.
- ObjectOutputStream: convierte objetos y variables en vectores de bytes que pueden ser escritos en un OutputStream.
- DataOutputStream, que da formato a los tipos primitivos y objetos String, convirtiéndolos en un flujo de forma que cualquier DataInputStream, de cualquier máquina, los pueda leer. Todos los métodos empiezan por "write", como writeByte(), writefloat(), etc.

De InputStream, para lectura de ficheros binarios, destacamos:

- FileInputStream: lee bytes de un fichero.
- ObjectInputStream: convierte en objetos y variables los vectores de bytes leídos de un InputStream.

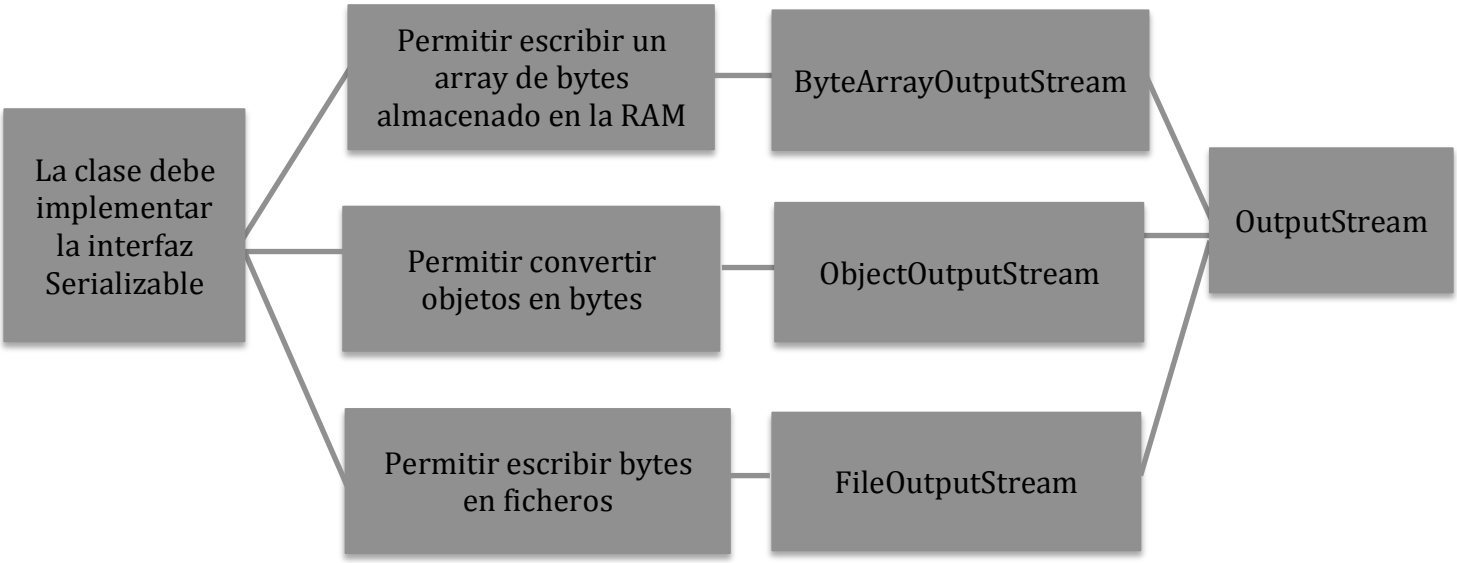
Constructores InputStream



- ByteArrayInputStream:
  - ✓ ByteArrayInputStream(byte[] buf)
  - ✓ ByteArrayInputStream(byte[] buf, int offset, int lenght)
- ObjectInputStream:
  - ✓ ObjectInputStream(InputStream in)
- FileInputStream:
  - ✓ FileInputStream(String name)
  - ✓ FileInputStream(File file)

buf -> Buffer de entrada
offset -> Desplazamiento inicial dentro del buffer
lenght -> Número máximo de bytes leídos del buffer
in -> InputStream con los datos para leer
name -> Nombre de fichero
file -> Fichero de entrada

Constructores OutputStream



- ByteArrayOutputStream:
  - ✓ ByteArrayOutputStream(int size)
- ObjectOutputStream:
  - ✓ ObjectOutputStream(OutputStream output)
- FileOutputStream:
  - ✓ FileOutputStream(String name)
  - ✓ FileOutputStream(String name, boolean append)
  - ✓ FileOutputStream(File file)
  - ✓ FileOutputStream(File file, boolean append)

size -> Capacidad inicial
output -> OutputStream donde escribir los datos
name -> Nombre del fichero de salida
append -> opción para escribir al inicio o final de fichero
file -> Fichero de salida

Ejemplo convertir un objeto en un fichero

```
private static void guardarObjeto(File file){
        try{
            file.createNewFile();
        }catch (IOException ex){
            System.err.println("No se ha podido crear el fichero " + file.getName());
            System.err.println(ex.getMessage());
            System.exit(-1);
        }
        //1.Crear el objeto que deseamos guardar
        //Pedimos los datos de Persona por consola
        Scanner scanner= new Scanner(System.in);
        System.out.println("Introduce el id de la persona");
        int id=scanner.nextInt();
        scanner.nextLine();
        System.out.println("Introduce el nombre de la persona");
        String nombre=scanner.nextLine();
        System.out.println("Introduce la edad de la persona");
        int edad= scanner.nextInt();
        scanner.nextLine();
        System.out.println("Introduce la dirección de la persona");
        String direccion=scanner.nextLine();

        Persona persona= new Persona(id,nombre,edad,direccion);

        //2.Crear el fileOutStream para guardar información en el fichero -> flujo de datos de salida
        FileOutputStream fileOutputStream=null;
        try{
            fileOutputStream=new FileOutputStream(file);
        }catch(FileNotFoundException ex){
            System.err.println("No se ha encontrado el fichero: " + file.getName());
            System.err.println(ex.getMessage());
            System.exit(-2);
        }

        //Convertir el objeto en bytes
```

```
        ObjectOutputStream objectOutputStream=null;
        try{
            //Utiliza el fileOutputStream para saber donde escribir
            objectOutputStream=new ObjectOutputStream(fileOutputStream);
        }catch(IOException ex){
            System.err.println("No se ha podido crear el ObjectOutputStream:");
            System.err.println(ex.getMessage());
            System.exit(-3);
        }
        //Guardar los bytes en el fichero
        try{
            //Escribimos el objeto en el fichero ligado al FileOutputStream
            objectOutputStream.writeObject(persona);
        }catch (IOException ex){
            System.err.println("No se ha podido escribir en el fichero: " + file.getName());
            System.err.println(ex.getMessage());
            System.err.println(ex);
            System.exit(-4);
        }
        //Cerrar el fichero
        try{
            fileOutputStream.close();
            objectOutputStream.close();
        }catch (IOException ex){
            System.err.println("No se ha podido cerrar correctamente el fichero: " + file.getName());
            System.err.println(ex.getMessage());
            System.exit(-5);
        }

    }
```

Ejemplo convertir un fichero en un objeto

```
private static void recuperarObjeto(File file){
        FileInputStream fileInputStream=null;
        try {
            //De donde debe leer
            fileInputStream=new FileInputStream(file);
        } catch (FileNotFoundException ex) {
            System.err.println("Error al abrir el FileInputStream");
            System.err.println(ex.getMessage());
            System.exit(-6);
        }
        ObjectInputStream objectInputStream=null;
        try {
            //Utiliza el FileInputStream para saber de donde leer
            objectInputStream=new ObjectInputStream(fileInputStream);
        } catch (IOException ex) {
            System.err.println("Error al crear el ObjectInputStream");
            System.err.println(ex.getMessage());
            System.exit(-7);
        }
        try {
            //Lee el fichero y lo convierte en una persona
            //IMPORTANTE:Hacer el casteo ->(Persona)
            Persona persona=(Persona) objectInputStream.readObject();
            System.out.println(persona);
        } catch (IOException | ClassNotFoundException ex) {
            System.err.println("Error al recuperar el objeto del fichero");
            System.err.println(ex.getMessage());
            System.exit(-8);
        }
        try {
            objectInputStream.close();
            fileInputStream.close();
        } catch (IOException ex) {
            System.err.println("Error al cerrar los InputStream");
            System.err.println(ex.getMessage());
```

```
            System.exit(-9);
        }
    }
```

En la clase Main implementaríamos lo siguiente:

```
    public static void main(String[] args) {
        //Crear el fichero donde guardaremos el fichero
        File file=new File("ficheros/objeto");
        guardarObjeto(file);
        recuperarObjeto(file);
    }
```

Y también deberíamos implementar la clase Persona.

```
package ficherosBinarios;

import java.io.Serializable;

public class Persona implements Serializable{
    private int id;
    private String nombre;
    private int edad;
    private String direccion;

    public Persona(int id, String nombre, int edad, String direccion) {
        this.id = id;
        this.nombre = nombre;
        this.edad = edad;
        this.direccion = direccion;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
```

```
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getEdad() {
        return edad;
    }
    public void setEdad(int edad) {
        this.edad = edad;
    }
    public String getDireccion() {
        return direccion;
    }
    public void setDireccion(String direccion) {
        this.direccion = direccion;
    }
    @Override
    public String toString() {
        return "Persona [id=" + id + ", nombre=" + nombre + ", edad=" + edad + ", direccion=" + direccion
+ "]";
    }
}
```

NOTA: Para guardar un objeto en un fichero la clase que deseamos guardar o el objeto que deseamos guardar debe implementar la interfaz Serializable .

NOTA: Existen diferentes tipos de Exceptions que deben ser controladas:

- IOException: Excepción producida al leer o escribir datos.
- FileNotFoundException: Excepción producida cuando no encuentra el fichero solicitado .
- ClassNotFoundException: Excepción producida cuando la JVM no es capaz de recuperar la clase que se está indicando.

Indica si la afirmación es verdadera o falsa:
Podemos escribir datos binarios a ficheros utilizando el método append de la clase DataOutputStream.

c. Verdadero
d. Falso

Streams de texto

Los streams de 16 bits, 2 bytes, orientados a caracteres utilizan dos clases abstractas que ya están preparadas para esta lectura de datos.

- Reader
- Writer

Estas dos clases están optimizadas para trabajar con caracteres y con texto en general, debido a que tienen en cuenta que cada carácter Unicode está representado por dos bytes.

Las subclases de Writer y Reader que permiten trabajar con ficheros de texto son:

- FileReader, para lectura desde un fichero de texto. Crea un flujo de entrada que trabaja con caracteres en vez de con bytes.
- FileWriter, para escritura hacia un fichero de texto. Crea un flujo de salida que trabaja con caracteres en vez de con bytes.

También se puede montar un buffer sobre cualquiera de los flujos que definen estas clases:

- BufferedWriter se usa para montar un buffer sobre un flujo de salida de tipo FileWriter.
- BufferedReader se usa para montar un buffer sobre un flujo de entrada de tipo FileReader.

Al crear un objeto de la clase FileWriter se puede pasar un segundo parámetro de tipo boolean, este especificará si escribirá en el fichero desde el inicio (false) o continuará desde el último punto (true).

Constructores FileWriter

- FileWriter(String name)
- FileWriter(String name, boolean append)
- FileWriter(File file)
- FileWriter(File file, boolean append)
- FileWriter(String name, Charset charset)
- FileWriter(String name, Charset charset, boolean append)
- FileWriter(File file, Charset charset)
- FileWriter(File file, Charset charset, boolean append)

Constructores FileReader

- FileReader(String name)
- FileReader(String File)
- FileReader(String name, Charset charset)
- FileReader(File file, Charset charset)

charset -> Es la codificación de los caracteres. Nos permite leer caracteres que de forma inicial no están previstos como símbolos de otros lenguajes.
output -> OutputStream donde escribir los datos
name -> Nombre del fichero de salida
append -> opción para escribir al inicio o final de fichero
file -> Fichero de salida

Señala si la afirmación es verdadera o falsa:
La clase Reader está optimizada para trabajar con ficheros binarios.

a. Verdadero
b. Falso

Ejemplo escribir en un fichero

```
File file=new File("./ficheros/ficherocaracteres.txt");
try {
     FileWriter fileWriter= new FileWriter(file);
     fileWriter.write("Hola mundo!!");
     fileWriter.close();
} catch (IOException e) {
     System.err.println("Error de apertura/escritura" + file.getName());
}
```

Ejemplo leer de un fichero

```
File file=new File("./ficheros/ficherocaracteres.txt");
try {
     FileReader fileReader= new FileReader(file);
     int read;
     while ((read=fileReader.read())!=-1){
        System.out.print((char)read);
     }
     System.out.println();
     fileReader.close();
} catch (IOException e) {
     System.err.println("Error de apertura/lectura" + file.getName());
}
```

NOTA: La función read, lee en cada vuelta del bucle 2 bytes, correspondientes a 1 carácter. Esta función realiza la lectura en número enteros por lo que debemos convertirlo en un carácter. Esta última acción se puede hacer con un simple casteo ya que cada número entero tiene una representación en forma de carácter. Ver equivalencias en la tabla ASCII https://elcodigoascii.com.ar

Gestión de flujos de datos - Streams – Acceso Aleatorio

A menudo, no necesitas leer un fichero de principio a fin, sino simplemente acceder al fichero como si fuera una base de datos, donde se salta de un registro a otro; cada uno en diferentes partes del fichero. Java proporciona una clase RandomAccessFile para este tipo de entrada/salida.

RandomAccessFile: Permite leer y escribir sobre el fichero, no es necesario dos clases diferentes.
Necesita que le especifiquemos el modo de acceso al construir un objeto de esta clase: sólo lectura o bien lectura y escritura.
Posee métodos específicos de desplazamiento como seek(long posicion) o skipBytes(int desplazamiento) para poder movernos de un registro a otro del fichero, o posicionarnos directamente en una posición concreta del fichero.

Funciones/métodos más importantes de RandomAccesFile:
- getFilePointer: Devuelve la posición del puntero.
- seek(long pos): Establece la posición del puntero.
- length(): Devuelve el tamaño del fichero en bytes.
- skipBytes(int salto): Desplaza el puntero "salto" posiciones.

Por esas características que presenta la clase, un archivo de acceso directo tiene sus registros de un tamaño fijo o predeterminado de antemano. Con esta clase, podemos acceder de forma aleatoria a un archivo o fichero. Además, los archivos de este tipo, se pueden leer, o bien leer y escribir a la vez.

Por otra parte, estos archivos no son stream y se numeran por un índice que empieza por cero. Este índice, se llama puntero o cursor de lectura/escritura e indica la posición a partir de la cual se empezará a leer o escribir en el archivo. Es importante indicar que la información almacenada en este tipo de archivos, se guarda en forma de bytes. Esto quiere decir que este la clase RandomAccessFile trata del archivo como un array o arreglo de bytes.

La clase posee dos constructores, que se comentan a continuación.

Constructores RandomAccessFile

- RandomAccessFile(String name, String accessMode)
- RandomAccessFile(File file, String accessMode)

name -> Nombre del fichero a abrir.
accessMode -> Modo de acceso al fichero. (r:lectura, rw:lectura y escritura)
file -> Fichero a abrir

Ejemplo RandomAccesFile:

```
File file = new File("teoria/aleatorio.txt");
try {
    RandomAccessFile randomAccessFile = new RandomAccessFile(file, "rw");
    System.out.println("Posición del puntero: " + randomAccessFile.getFilePointer());
    String texto = "Linea 1\nLinea 2";
    randomAccessFile.seek(0);
    randomAccessFile.write(texto.getBytes(StandardCharsets.UTF_8));
    System.out.println("Posición del puntero: " + randomAccessFile.getFilePointer());
    randomAccessFile.seek(0);
    String line;
    while ((line= randomAccessFile.readLine())!=null){
        System.out.println("Texto leído del fichero:");
        System.out.println(line);
    }
    System.out.println("Posición del puntero: " + randomAccessFile.getFilePointer());
    randomAccessFile.seek(2);
    System.out.println("Posición del puntero: " + randomAccessFile.getFilePointer());
    randomAccessFile.close();
} catch (FileNotFoundException e) {
    System.err.println("No se ha encontrado el fichero: " + file.getName());
    e.printStackTrace();
} catch (IOException e) {
    System.err.println("Error al recuperar o insertar información");
    e.printStackTrace();
}
```

Enlaces de interés sobre ficheros:

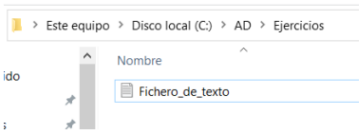
https://docs.oracle.com/javase/tutorial/essential/io/
https://www.tutorialspoint.com/java/java_files_io.htm
https://docs.oracle.com/javase/8/docs/technotes/guides/io/enhancements.html

Señala si la afirmación es verdadera o falsa:
Un objeto de la clase RandomAccessFile necesita el modo de acceso al crear el objeto.

c. Verdadero
d. Falso

EJERCICIOS FICHEROS SECUENCIALES

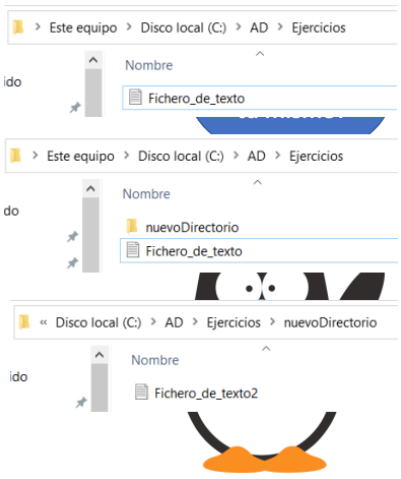
1. Crea un programa en Java que cree un fichero de texto llamado `Fichero_de_texto.txt` en la ruta `C:\AD\Ejercicios`



2. Añadir al ejercicio anterior un menú de opciones al programa anterior y que el usuario pueda elegir qué hacer:

- 1.Crear un directorio llamado `nuevoDirectorio` en la ruta `C:\AD\Ejercicios`, dentro de él se creará un nuevo fichero de texto llamado `Fichero_de_texto2.txt`
- 2.Eliminar el fichero `Fichero_de_texto.txt` que creamos en el ejercicio anterior
- 3.Eliminar el directorio `nuevoDirectorio`
- 4.Salir

Las opciones se podrán seguir eligiendo hasta que el usuario elija la opción de salir.



3. Crea un programa que a partir de una cadena de caracteres permita almacenarlos en un fichero de texto. El programa añadirá un carácter `*` para indicar que ha llegado al final del fichero.
4. Variante del ejercicio anterior que almacene Strings a partir de un array.

- Usa un array de String.
- Guarda cada elemento del array en un fichero (una línea por cada String).
- Añade al final del fichero un carácter * para indicar el fin.

5. Ahora el programa deberá leer el contenido del fichero creado en el primer punto y lo mostrará por consola.
6. Crea un programa en java que almacene en un fichero de texto llamado Empleados.txt un listado con 5 empleados, indicando en cada linea el ID y el nombre de cada empleado. Nota: Los empleados y los Ids puedes introducirlos desde teclado o desde código, como quieras.
7. A continuación el programa anterior deberá leer el contenido del fichero Empleados.txt y mostrarlo por pantalla.

EJERCICIOS FICHEROS ALEATORIOS

8. Crear un programa en Java que inserte datos de empleados en un fichero aleatorio `RandomAccessFile`, llamado `empleados.dat`, indicando apellido, departamento y salario. Además para cada empleado se insertará un identificador (mayor que cero) y coincidirá con el índice +1 que recorren los arrays.

Resumiendo para cada empleado se guarda:

- **Identificador** (4 bytes, int)
- **Apellido** (10 caracteres → 20 bytes, char en UTF-16 / Unicode)
- **Departamento** (4 bytes, int)
- **Salario** (8 bytes, double)

Luego el tamaño del registro será de 36 bytes: 4bytes + 20bytes + 4bytes + 8bytes

Partimos de estos arrays de ejemplo:

```
String[] apellidos = {"Lopez", "Garcia", "Perez", "Santos", "Diaz"};
int[] departamentos = {10, 20, 10, 30, 10};
double[] salarios = {1500.50, 2100.75, 1800.00, 1750.30, 1600.00};
```

9. Crea un programa en Java que muestre los datos introducidos en el fichero del ejercicio anterior, `empleados.dat`. El posicionamiento para empezar a recorrer es 0 y para recuperar los datos hay que añadir 36 a la variable de posicionamiento.

Ten en cuenta el tamaño del cada registro y la longitud total del fichero para obtener el número de registros del fichero.

El resultado obtenido por pantalla será el siguiente:

```
=== Lista de Empleados ===

ID: 1 | Apellido: Lopez | Departamento: 10 | Salario: 1500,50
ID: 2 | Apellido: Garcia | Departamento: 20 | Salario: 2100,75
ID: 3 | Apellido: Perez | Departamento: 10 | Salario: 1800,00
ID: 4 | Apellido: Santos | Departamento: 30 | Salario: 1750,30
ID: 5 | Apellido: Diaz | Departamento: 10 | Salario: 1600,00
```

10. Crea un programa en Java que consulte los datos de un empleado en particular del fichero creado en el ejercicio anterior, `empleados.dat`.
    a. Por ejemplo: busca el empleado con ID 4

Esta es la secuencia de lo que se pedirá y mostrará por pantalla.

```
Introduce el ID del empleado a buscar: 4

Empleado encontrado:
ID: 4
Apellido: Santos
Departamento: 30
Salario: 1750,30
```

11. Crea un programa en Java que inserte los datos de un empleado nuevo en el fichero creado anteriormente, `empleados.dat`. Pasos para realizar el programa:

    - Pide al empleado por consola:

        - Apellido (rellena hasta 10 caracteres)
        - Departamento (int)
        - Salario (double)

    - Calcula el nuevo ID (último ID "registro" + 1)
    - Escribe el nuevo registro al final del archivo `empleados.dat`

EJERCICIOS BLOQUE 2

1. Crea una nueva clase llamada Persona (Persona.java) con los atributos (id, nombre, edad y dni). Recuerda añadir a la clase persona los setters y getters, además del método toString () que representa al objeto.
    a. Crea una función para guardar un objeto Persona en un fichero con el nombre persona1
    b. Crea una función para recuperar un objeto Persona del fichero persona1
    c. Modifica sus propiedades y vuelve a guardarlo en el fichero persona 1
2. Crea un fichero de texto utilizando la clase FileWriter
    a. El fichero debe contener la información: “Esto es un texto de prueba, Estamos creando nuestro primer fichero de texto https://codigonline.com”
3. Lee un fichero de tipo imagen y muestra su contenido por pantalla.
    a. ¿Se puede leer de forma correcta?

A continuación se muestra como tiene que ser la estructura de la clase Main.java:

```
//Aquí las librerías que necesites importar para trabajar con ellas


public class Main {
          // Método para guardar un objeto de tipo Persona en un fichero
          public static void guardarObjeto(Persona persona, String filename) {
          }

          // Método para recuperar un objeto de tipo Persona de un fichero
          public static Persona recuperarObjeto(String filename) {
          }

          // Main
          public static void main(String[] args) {
          //1.a. Crea una función para guardar un objeto Persona en un fichero con el nombre persona1
                    // Creamos el objeto Persona

                    //Guardamos el objeto Persona en un fichero

         // 1.b. Crea una función para recuperar un objeto Persona del fichero persona1
                    //  Recuperamos el objeto Persona del fichero

         // 1.c. Modifica sus propiedades y vuelve a guardarlo en el fichero persona1
```

```
                    //  Recuperamos de nuevo el objeto Persona del fichero para ver los cambios


        // 2.a. Crear un fichero de texto utilizando FileWriter


        // 3.a Lee un fichero de tipo imagen (binario), p.e: crondose.jpg puede ser cualquiera, y muestra
        su contenido por pantalla


        }
}
```

Ficheros XML

¿Qué es un fichero XML?

XML: Las siglas de XML vienen de eXtensible Markup Language (Lenguaje de marcado extendido).

Este nos permite crear estructuras anidando los elementos unos dentro de otros y definir el contenido de cada elemento.

Características:

- Los documentos XML son muy fáciles de leer, ya que tienen una sintaxis muy sencilla.
- Están basados en etiquetas que tienen un nombre y a su vez pueden tener atributos.
- Son muy usados en ficheros de configuración de algunos programas o protocolos SOAP para enviar información a los servidores y ejecutar diferentes rutinas.

¿Cómo se trabaja con datos XML desde el punto de vista del desarrollador de aplicaciones?

Una aplicación que consume información XML debe:

- Leer un fichero de texto codificado según dicho estándar.
- Cargar la información en memoria y, desde allí...
- Procesar esos datos para obtener unos resultados (que posiblemente también almacenará de forma persistente en otro fichero XML).

El proceso anterior se enmarca dentro de lo que se conoce en informática como "parsing" o análisis léxico-sintáctico, y los programas que lo llevan a cabo se denominan "parsers" o analizadores (léxico-sintácticos). Más específicamente podemos decir que el "parsing XML" es el proceso mediante el cual se lee y se analiza un documento XML para comprobar que está bien formado para, posteriormente, pasar el contenido de ese documento a una aplicación cliente que necesite consumir dicha información.

Schema: Un esquema (o schema) es una especificación XML que dicta los componentes permitidos de un documento XML y las relaciones entre los componentes. Por ejemplo, un esquema identifica los elementos que pueden aparecer en un documento XML, en qué orden deben aparecer, qué atributos pueden tener, y qué elementos son subordinados (esto es, son elementos hijos) para otros elementos. Un documento XML no tiene por qué tener un esquema, pero si lo tiene, debe atenerse a ese esquema para ser un documento XML válido. NOTA: Ver diferencia con DTD en Anexo.

Señala si la afirmación es verdadera o falsa:
XML hace más fácil el intercambio de información entre sistemas.

a. Verdadero
b. Falso

Un XML se compone de la declaración del XML:

```
<?xml versión= “1.0” encoding= “UTF-8”?>
```

En esta parte se declara el XML, la versión del documento y se define el *encoding* que se utilizará en el fichero. Debemos definir esta línea como nuestra primera línea de cualquier fichero XML. Los campos *versión* y *encoding* deben estructurarse en este orden para considerarse una estructura correcta. De todos modos, las declaraciones son opcionales, pero si se establece *encoding* se deberá añadir también la declaración *versión*. La versión nos servirá para indicar en qué momento se realizó el documento y seguir una evolución del estándar si se modifica el archivo en un futuro. El *encoding*, como hemos explicado en apartados anteriores, nos permitirá definir qué caracteres vamos a utilizar. Por defecto, utilizaremos UTF-8.

A continuación, añadiremos el cuerpo del XML, que es la parte más importante del fichero. Este documento adquiere una estructura de árbol, compuesto por un elemento raíz o principal dentro del cual añadiremos el resto de los elementos.

```
<?xml version="1.0" encoding="UTF-8"?>
<coches>
          <coche>
                    <marca>Seat</marca>
                    <modelo>Ibiza</modelo>
                    <color>rojo</color>
                    <matriculacion>2019</matriculacion>
          </coche>
          <coche>
                    <marca>Ford</marca>
                    <modelo>Focus</modelo>
                    <color>gris</color>
                    <matriculacion>2014</matriculacion>
          </coche>
</coches>
```

NOTA: Algo muy importante de XML es que sigue una jerarquía en forma de árbol, por lo que todo elemento (excepto la ráiz) tiene un único padre, pero cada elemento puede tener muchos hijos.
De la misma forma no se pueden dar elementos cíclicos, un elemento no puede ser padre e hijo de otro elemento.

¿Cómo están formados?

Están formados por:

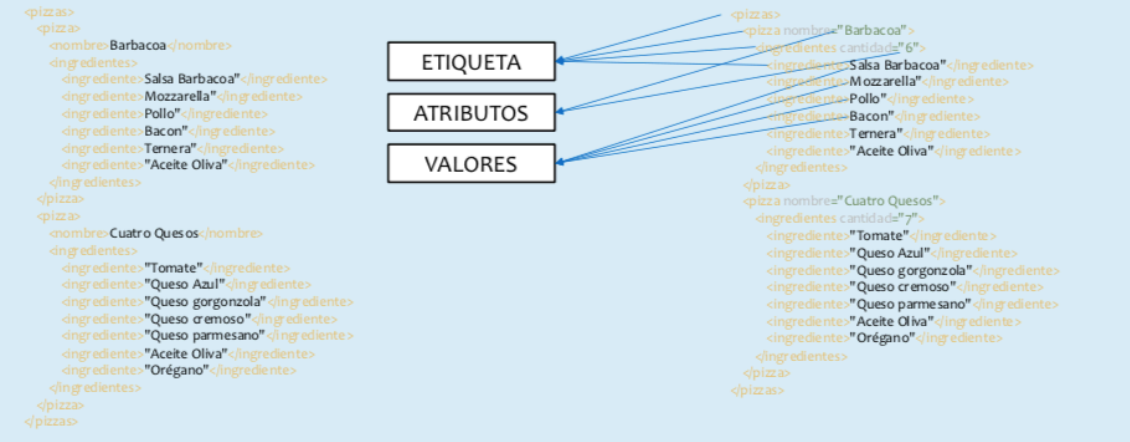
- Etiquetas XML
- Atributos XML

Etiquetas XML

Son obligatorias. Cada etiqueta XML tendrá un inicio que se definirá dentro de los símbolos "<etiqueta>".
De la misma forma tendrá un fin que se definirá dentro de los símbolos "</etiqueta>".
Entre la etiqueta de inicio y de fin pueden ir otras etiquetas creando así la estructura.

Atributos XML

Son opcionales. A su vez una etiqueta puede tener 0..* atributos, elementos opcionales que dotan al XML de mayor información.
Algo muy importante es que toda información en atributos puede ser representada en elementos, mientras que no todos los elementos pueden ser atributos.
¿Cómo saber donde va la información?
La única forma de determinarlo es saber si el elemento estará repetido y si tendrá hijos. En estos dos casos la información no puede ir en atributos.



¿Cómo leer ficheros XML?

Para leer un fichero XML necesitamos hacer uso de un **parser**

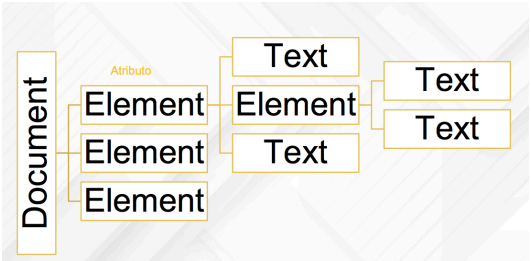
Un **parser** es un analizador sintáctico para XML que se encarga de verificar que la estructura de ese fichero de texto es correcta.

Existe formas diferentes de poder leer ficheros XML, ver su estructura y atributos. Estas herramientas son conocidas como XML-parser. Las más utilizadas son DOM y SAX:

- DOM: Document Object Model, el procesador lee todo el documento XML y lo almacena en memoria RAM. Es muy útil cuando queremos acceder de forma rápida a un elemento del árbol, ya que tiene toda la información precargada (similiar al acceso de un fichero aleatorio). Contra más grande es el documento más tiempo y memoria necesita para procesarlo.
- SAX: Simple Api for XML, el procesador va leyendo de forma secuencial el fichero y va lanzando eventos que nuestro programa debe capturar. Para cada etiqueta de inicio/fin, atributo y valor emitirá un evento. Es mucho más rápido que DOM y consume menos memoria. En contra si queremos acceder a un elemento en concreto debemos de recorrer todo el documento.

Característica	SAX (Simple API for XML)	DOM (Document Object Model)
Modo de funcionamiento	Basado en eventos (lectura secuencial). El parser lee el XML de principio a fin y lanza eventos (inicio de etiqueta, fin de etiqueta, texto, etc.).	Carga todo el XML en memoria y construye una estructura jerárquica (árbol de nodos).
Memoria	Muy baja: solo mantiene en memoria los datos que está procesando en ese momento.	Alta: todo el documento se almacena en memoria.
Velocidad	Más rápido para leer documentos grandes (porque no necesita construir ni mantener un árbol completo).	Más lento en carga inicial, ya que debe construir toda la estructura.
Acceso aleatorio	No posible directamente (lectura solo secuencial).	Sí, puedes acceder fácilmente a cualquier nodo.
Complejidad de uso	Más complejo: el programador debe manejar los eventos.	Más sencillo: puedes navegar por el árbol con métodos de acceso.

DOM (Document Object Model)



DOM es una plataforma e interfaz de lenguaje estándar que permite a los programas y a los scripts acceder y actualizar dinámicamente el contenido, la estructura y el estilo de un documento. Se caracteriza por tener una estructura en forma de árbol.

Clases más importantes del DOM:

- DocumentBuilderFactory: Es una clase especial, nos da la capacidad de poder crear parsers para nuestro programa.
- DocumentBuilder: Define el parser DOM que se va a utilizar.
- Document: Objeto que contiene la lectura completa del XML.
- Node: Representa a cualquier Nodo del árbol.
- NodeList: Lista que contiene todos los nodos hijos de un nodo.
- Element: Es un tipo de nodo, representa un elemento del XML.
- Attr: Representa un atributo de un Nodo.
- Text: Representa el texto de un elemento.

Gracias a estas clases podemos leer/escribir documentos XML utilizando DOM.

Métodos DOM más usuales:

- Document.getDocumentElement() – Retorna el elemento raiz del documento.
- Node.getFirstChild() – Retorna el primer nodo hijo del nodo.
- Node.getLastChild() – Retorna el ultimo nodo hijo del nodo.
- Node.getNextSibling() – Retorna el siguiente hermano de un nodo.
- Node.getPreviousSibling() – Retorna el hermano anterior de un nodo.
- Node.getAttribute(attrName) – Retorna el atributo del nodo con el nombre que se pasa como atributo.

Leer Archivos - DOM

Pasos para leer un documento:
1. Crear el DocumentBuilderFactory
2. Crear el DocumentBuilder
3. Crear el Document, este punto es muy importante ya que cargará todo el documento en memoria.
4. Leer las propiedades del documento deseadas. Este último paso variará según el documento y las propiedades que deseamos leer.

NOTA: Antes de empezar, debemos tener en cuenta que los espacios en blanco dentro de una etiqueta los lee como texto y debemos tener mucho cuidado al momento de tratar el fichero.
Se recomienda utilizar algún simplificador de XML como por ejemplo https://codebeautify.org/xmlviewer, donde podremos eliminar todos los espacios en blanco dándole al botón de Minify.

Ejemplo Lectura
(Para hacer pruebas utilizar este ejemplo de fichero XML, books.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<Catalog>
   <Book id="bk101" name="101">
      <Author>Garghentini, Davide</Author>
      <Title>XML Developer's Guide</Title>
      <Genre>Computer</Genre>
      <Price>44.95</Price>
      <PublishDate>2000-10-01</PublishDate>
      <Description>An in-depth look at creating applications with XML.</Description>
   </Book>
   <Book id="bk102" name="102">
      <Author>Garcia, Debra</Author>
      <Title>Midnight Rain</Title>
      <Genre>Fantasy</Genre>
      <Price>5.95</Price>
      <PublishDate>2000-12-16</PublishDate>
      <Description>A former architect battles corporate zombies, an evil sorceress, and her own childhood
to become queen of the world.</Description>
   </Book>
</Catalog>
```

A continuación se detalla el código para leer ese fichero XML y mostrar su información por pantalla:

```java
import java.io.File;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.DocumentBuilder;
import org.w3c.dom.Document;
import org.w3c.dom.NodeList;
import org.w3c.dom.Node;
import org.w3c.dom.Element;


public class DOMParseLecturaBOOK {
    public static void main(String[] args) {
        try {
```

```
            //**** 1. Obtiene el objeto doc que representa el documento en memoria
            //Define un objeto File para el archivo XML llamado books.xml.
            File inputFile = new File("books.xml");

            //Crea una instancia de DocumentBuilderFactory, que es la clase responsable de la
construcción del documento XML.
            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();

            //Se crea un DocumentBuilder, que será utilizado para analizar el archivo XML.
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();

            //El método parse() se utiliza para analizar el archivo XML y
            //devolver un objeto Document que representa la estructura del documento XML en memoria.
            Document doc = dBuilder.parse(inputFile);

            //Este paso asegura que el documento esté en una forma consistente,
            //eliminando nodos vacíos o innecesarios. Es un paso de "limpieza" del XML.
            //No es necesario pero es una buena práctica
            doc.getDocumentElement().normalize();

            //**** 2. Obtiene el nombre del elemento raíz del documento XML. Por ejemplo,
            //si el XML tiene una estructura como <Catalogo>...</Catalogo>, entonces esto devolverá
Catalogo.
            System.out.println("Root element :" + doc.getDocumentElement().getNodeName());

            //**** 3. Obtiene todos los nodos con el nombre de etiqueta Book dentro del XML, de la
siguiente forma:
            //Esto es importante ya que se asume que cada elemento Book contiene información sobre un
libro.
            NodeList nList = doc.getElementsByTagName("Book");
            System.out.println("----------------------------");

            //**** 4. Se recorre la lista de nodos Book. La variable temp es el índice del libro actual.
            for (int temp = 0; temp < nList.getLength(); temp++) {
                //Recupera el nodo Book actual.
                Node nNode = nList.item(temp);

                //Imprime el nombre del nodo actual, que debería ser Book.
                System.out.println("\nCurrent Element :" + nNode.getNodeName());

                //Verifica si el nodo actual es un nodo de tipo ELEMENT_NODE,
                //lo cual significa que es un nodo que contiene un elemento (en este caso, Book).
                if (nNode.getNodeType() == Node.ELEMENT_NODE) {
                    //Convierte el nodo a un Element, lo que permite acceder a sus atributos y
subelementos.
                    Element eElement = (Element) nNode;

                    //Obtiene el atributo id del elemento Book.
                    System.out.println("ID : "+ eElement.getAttribute("id"));
                    System.out.println("NAME : "+ eElement.getAttribute("name"));
```

```
                    //Accede al contenido del primer (y único) elemento Author dentro del elemento Book.
                    //Lo mismo aplica para los elementos Title, Genre, Price, PublishDate, y Description.
                    System.out.println("Author : "+
eElement.getElementsByTagName("Author").item(0).getTextContent());
                    System.out.println("Title : "+
eElement.getElementsByTagName("Title").item(0).getTextContent());
                    System.out.println("Genre :"+
eElement.getElementsByTagName("Genre").item(0).getTextContent());
                    System.out.println("Price : "+
eElement.getElementsByTagName("Price").item(0).getTextContent());
                    System.out.println("PublishDate :
"+eElement.getElementsByTagName("PublishDate").item(0).getTextContent());
                    System.out.println("Descripcion :
"+eElement.getElementsByTagName("Description").item(0).getTextContent());
                }
            }
            } catch (Exception e) {
                //Captura cualquier excepción que ocurra durante la ejecución del bloque try,
                //como errores al leer el archivo XML o problemas al analizar el contenido.
                e.printStackTrace(); //Imprime el stack trace para depurar el error.
                System.err.println(e); //Imprime el error en la consola.
        }

    }
}
```

Escribir Archivos – DOM

Escribir un documento XML con DOM es muy sencillo (pero bastante tedioso), únicamente debemos ir línea a línea estableciendo las características que deseamos.

Pasos para escribir un documento:
1. Crear el DocumentBuilderFactory
2. Crear el DocumentBuilder
3. Crear el DOMImplementation (Difiere de la lectura donde no hace falta)
4. Crear el Document, este punto es muy importante ya que cargará todo el documento en memoria.
5. Escribir las propiedades deseadas.
6. Transformar el DOM en memoria en un archivo del disco.
    - Se necesita un objeto Source (origen de los datos) y un Result (destino de los datos)
    - Se enviará del origen al destino mediante la clase Transform.

Ejemplo Escritura

A continuación se muestra el código para escribir en un documento XML, libros2.xml

```
import java.io.File;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import javax.xml.transform.Transformer;
import javax.xml.transform.TransformerException;
import javax.xml.transform.TransformerFactory;
import javax.xml.transform.OutputKeys;
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;

import org.w3c.dom.Attr;
import org.w3c.dom.Document;
import org.w3c.dom.Element;

public class DOMParseEscrituraBOOK {

  public static void main(String argv[]) {
    try {
        //**** 1. Se crea el documento para ello:
        //Se crea una instancia de DocumentBuilderFactory, que es la fábrica para construir objetos
DocumentBuilder.
        //Este objeto permitirá construir un documento XML en memoria.
        DocumentBuilderFactory docFactory = DocumentBuilderFactory.newInstance();

        //Se obtiene un DocumentBuilder a partir de la fábrica. Este se utiliza para crear un nuevo
documento XML.
        DocumentBuilder docBuilder = docFactory.newDocumentBuilder();
```

```
        //Se crea un nuevo documento XML vacío en memoria.
        //Este documento será la estructura que se va a escribir en un archivo XML.
        Document doc = docBuilder.newDocument();

        //**** 2. Se crea el elemento ROOT ó RAIZ para ello:
        //Se crea un nuevo elemento llamado rootElement (le puedes llamar como quieras), que será el
elemento raíz del XML.
        Element rootElement = doc.createElement("Catalogo");

        //Se agrega el elemento root como el hijo principal del documento, convirtiéndose en el nodo raíz
del XML.
        doc.appendChild(rootElement);

        //**** 3. Se crea un hijo del nodo root ó raiz para ello:
        //Se crea un nuevo elemento llamado libro.
        Element libro = doc.createElement("Libro");
        //Se añade el libro como hijo del nodo raíz (rootElement).
        rootElement.appendChild(libro);
        //**** **** 3.1 Se pueden crear atributos asociados al elemento libro, para ello:
        //Se crea un atributo llamado id.
        Attr attr = doc.createAttribute("id");
        //Se asigna un valor al atributo id.
        attr.setValue("bk101");
        //Se asocia el atributo id con el nodo libro.
        //Esto significa que el elemento libro tendrá un atributo id con el valor "valor del atributo".
        libro.setAttributeNode(attr);

        //NOTA: Aqui se podrían crear más atributos para el elemento libro de la misma forma
        Attr attr3 = doc.createAttribute("isbn");
        //attr = doc.createAttribute("isbn");
        attr3.setValue("01");
        libro.setAttributeNode(attr3);

        //**** **** 3.2 Se crean hijos del elemento libro, para ello:
        Element autor=doc.createElement("Autor");
        autor.setTextContent("Garghentini, Davide");

        Element titulo=doc.createElement("Titulo");
        titulo.setTextContent("XML Developer's Guide");

        Element genero=doc.createElement("Genero");
        genero.setTextContent("Computer");

        Element precio=doc.createElement("Precio");
        precio.setTextContent("44.95");

        Element fechapublicacion=doc.createElement("FechaPublicacion");
        fechapublicacion.setTextContent("2000-10-01");
```

```
        Element descripcion=doc.createElement("Descripcion");
        descripcion.setTextContent("An in-depth look at creating applications with XML.");

        //Se agregan los elementos como hijos del elemento libro.
        libro.appendChild(autor);
        libro.appendChild(titulo);
        libro.appendChild(genero);
        libro.appendChild(precio);
        libro.appendChild(fechapublicacion);
        libro.appendChild(descripcion);

        //**** 4. Se crea otro hijo del nodo root ó raiz para ello:
        //Se crea un nuevo elemento llamado libro2.
        Element libro2 = doc.createElement("Libro");
        //Se añade el libro2 como hijo del nodo raíz (root).
        rootElement.appendChild(libro2);
        //**** **** 4.1 Se pueden crear atributos asociados al elemento libro2, para ello:
        // Se crea un atributo llamado id.
        Attr attr2 = doc.createAttribute("id");
        //Se asigna un valor al atributo id.
        attr2.setValue("bk102");
        //Se asocia el atributo id con el nodo libro2.
        //Esto significa que el elemento libro2 tendrá un atributo id con el valor "valor del atributo".
        libro2.setAttributeNode(attr2);

        //NOTA: Aqui se podrían crear más atributos para el elemento libro2 de la misma forma
        attr2 = doc.createAttribute("isbn");
        attr2.setValue("02");
        libro2.setAttributeNode(attr2);

        //**** **** 4.2 Se crean hijos del elemento2, para ello
        Element autor2=doc.createElement("Autor");
        autor2.setTextContent("Garcia, Debra");

        Element titulo2=doc.createElement("Titulo");
        titulo2.setTextContent("Midnight Rain");

        Element genero2=doc.createElement("Genero");
        genero2.setTextContent("Fantasy");

        Element precio2=doc.createElement("Precio");
        precio2.setTextContent("5.95");

        Element fechapublicacion2=doc.createElement("FechaPublicacion");
        fechapublicacion2.setTextContent("2000-12-16");

        Element descripcion2=doc.createElement("Descripcion");
        descripcion2.setTextContent("A former architect battles corporate zombies,an evil sorceress, and
her own childhood to become queen of the world.");
```

```
        //Se agregan los elementos como hijos del elemento libro2.
        libro2.appendChild(autor2);
        libro2.appendChild(titulo2);
        libro2.appendChild(genero2);
        libro2.appendChild(precio2);
        libro2.appendChild(fechapublicacion2);
        libro2.appendChild(descripcion2);

        //**** 5. Se escribe el contenido del XML en un archivo, para ello:
        //Se crea un objeto factory para construir objetos Transformer,
        //que se encargan de transformar el contenido DOM en un formato XML legible.
        TransformerFactory transformerFactory = TransformerFactory.newInstance();

        //Se obtiene un Transformer que se encargará de transformar el documento DOM en un archivo XML.
        Transformer transformer = transformerFactory.newTransformer();
        //La siguiente línea se utiliza para configurar el formato de salida de un documento XML cuando se
utiliza un transformador
        //esta línea establece que la salida XML debe estar "indentada" (con sangría), lo que facilita la
lectura del XML generado.
        transformer.setOutputProperty(OutputKeys.INDENT, "yes");
        //Se establece la fuente de datos, que en este caso es el documento XML en memoria (doc).
        DOMSource source = new DOMSource(doc);

        //Define el archivo de salida donde se escribirá el XML.
        //Aquí, el archivo se guardará en la carpeta base con el nombre libros2.xml.
        StreamResult result = new StreamResult(new File("libros2.xml"));

        //Se realiza la transformación del documento DOM a XML y se guarda en el archivo especificado.
        transformer.transform(source, result);
        System.out.println("Fichero XML creado con éxito");

    } catch (ParserConfigurationException pce) { //Captura los errores relacionados con la configuración
del parser DOM.
        pce.printStackTrace();
    } catch (TransformerException tfe) { //Captura los errores que puedan ocurrir durante la
transformación del documento a formato XML.
        tfe.printStackTrace();
    }
  }
}
```

SAX (API Simple para XML)

Los parsers de tipo SAX funcionan de forma diferente a los de DOM, van leyendo el documento poco a poco y cada vez que encuentren un elemento, atributo, texto, etc., lanzarán un evento para que lo capturemos y podamos actuar en consecuencia.

La API de SAX es mucho más compleja que la de DOM pero vamos a analizarla.

Clases más importantes de SAX:

- SAXParserFactory: Clase especial, nos permitirá crear nuevos SAX parsers
- SAXParser: Creación de un nuevo parser para SAX, utiliza la clase anterior
- XMLReader: Objeto que permite leer el documento XML elemento a elemento
- DefaultHandler: Clase abstracta que debemos implementar, contiene las llamadas a los eventos.

La clase DefaultHandler entre otras, contiene las siguientes funciones que son las que más nos interesan:

- startDocument(): Se llama cuando se detecta que el documento empieza. Aquí deben indicarse las acciones a realizar al inicio del documento.
- endDocument():Se llama cuando se detecta que el documento ha acabado. Por lo tanto, aquí deben indicarse las acciones a realizar al finalizar el documento.
- startElement(): Se llama cuando encuentra un nuevo elemento, nodo, etiqueta, tag, etc. Aquí debe indicarse el tratamiento que se deberá realizar sobre cada nuevo elemento, como la recogida de información de sus atributos.
- endElement(): Se llama una vez ha leído el elemento. Aquí se encuentra la información del nodo y del contenido del nodo por lo que es aquí donde, normalmente, se recoge la información.
- characters(): Se invoca para encontrar una cadena de texto.

Leer Archivos - SAX

Para leer un fichero XML con SAX seguiremos los siguientes pasos:

1. Crear SAXParserFactory.
2. Crear el SAXParser.
3. Crear el XMLReader.
4. Implementar el DefaultHandler.
    a. Dar funcionalidad a las diferentes funciones según nuestros requisitos.
5. Parsear el documento .

```
import org.xml.sax.Attributes;
import org.xml.sax.SAXException;
import org.xml.sax.helpers.DefaultHandler;

import javax.xml.parsers.SAXParser;
import javax.xml.parsers.SAXParserFactory;
import java.io.File;
```

```java
public class SAXLeerXml {

    public static void main(String[] args) {
        try {
            // Crear el parser SAX
            SAXParserFactory factory = SAXParserFactory.newInstance();
            SAXParser saxParser = factory.newSAXParser();

            // Crear el manejador de eventos
            BookHandler handler = new BookHandler();

            // Parsear el archivo XML
            saxParser.parse(new File("books.xml"), handler);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

// Clase manejadora de eventos SAX
class BookHandler extends DefaultHandler {

    /* StringBuilder → es una clase de Java que permite construir cadenas de texto (strings)
    de forma eficiente y modificable.
    valorActual → es el nombre de la variable. La usamos para ir acumulando el texto
    que el parser SAX encuentra dentro de las etiquetas XML. */
    private StringBuilder valorActual = new StringBuilder();
    private String id;

    private String author;
    private String title;
    private String genre;
    private String price;
    private String publishDate;
    private String description;

    // Se llama cuando comienza una etiqueta
    @Override
    public void startElement(String uri, String localName, String qName, Attributes attributes)
            throws SAXException {

        // Reiniciar valorActual para cada elemento
        //Vacía el contenido del StringBuilder antes de empezar a leer un
        //nuevo elemento XML.
        valorActual.setLength(0);

        if (qName.equalsIgnoreCase("Book")) {
            id = attributes.getValue("id");
            System.out.println("\n Libro encontrado:");
```

```
            System.out.println("ID: " + id);
        }
    }

    // Se llama cuando se encuentra texto entre etiquetas
    @Override
    public void characters(char[] ch, int start, int length) throws SAXException {
        valorActual.append(ch, start, length);
    }

    // Se llama cuando se cierra una etiqueta
    @Override
    public void endElement(String uri, String localName, String qName) throws SAXException {
        switch (qName) {
            case "Author":
                author = valorActual.toString().trim();
                break;
            case "Title":
                title = valorActual.toString().trim();
                break;
            case "Genre":
                genre = valorActual.toString().trim();
                break;
            case "Price":
                price = valorActual.toString().trim();
                break;
            case "PublishDate":
                publishDate = valorActual.toString().trim();
                break;
            case "Description":
                description = valorActual.toString().trim();
                break;
            case "Book":
                // Mostrar datos cuando termina el libro
                System.out.println("Autor: " + author);
                System.out.println("Título: " + title);
                System.out.println("Género: " + genre);
                System.out.println("Precio: " + price);
                System.out.println("Fecha publicación: " + publishDate);
                System.out.println("Descripción: " + description);
                break;
        }
    }
}
```

Señala si la afirmación es verdadera o falsa:
El SAX lee todo el fichero .xml y lo muestra

a. Verdadero
b. Falso

Binding

El Binding es una técnica que consiste en vincular clases Java con formatos específicos de almacenamiento de manera automatizada.

En Java existen varias bibliotecas para gestionar el binding , como por ejemplo JAXB , JiBX , XMLBinding , etc. Desde la versión 6.0 se ha incorporado en el JDK estándar JAXB, una potente biblioteca.

Java Architecture for XML Binding (JAXB) permite a los desarrolladores Java asignar clases de Java a representaciones XML. JAXB proporciona dos características principales: la capacidad de serializar las referencias de objetos Java a XML y la inversa, es decir, deserializar XML en objetos Java. En otras palabras, JAXB permite almacenar y recuperar datos en memoria en cualquier formato XML, sin la necesidad de implementar un conjunto específico de rutinas de carga y guardado de XML para la estructura de clases del programa.

Configuración con anotaciones

Las Anotaciones pueden asociarse a un paquete, a una clase, a un atributo o incluso a un parámetro. Estas clases especiales se declaran en el código de la aplicación anteponiendo el símbolo @ en el nombre de la Anotación . Cuando el compilador de Java detecta una Anotación crea una instancia y la inyecta en el elemento estructural afectado (paquete, clase, método, atributo, etc.). Esto hace que éstas no aparezcan como atributos o métodos propios del objeto, y por eso decimos que no interacciona con el modelo de datos, pero las aplicaciones que lo necesiten pueden obtener la instancia inyectada y usarla.

Las Anotaciones pueden declararse con parámetros o sin ellos. En caso de que tengan parámetros, estos pueden ser otros Anotaciones, valores constantes o valores literales, de modo que estén disponibles en tiempo de compilación (que es cuando el compilador realiza la inyección).

Anotaciones principales.

- @XmlRootElement(namespace = "namespace", name = "nombre"): Define la raíz del XML.
- @XmlType(propOrder = { "field2", "field1",.. }, name = "nombre"): Permite definir en que orden se van escribir los elementos dentro del XML. Esta anotación deberán tenerla todas las clases que no mapean la raiz del XML.
- @XmlElement(name = "nombre"): Define el elemento de XML que se va usar. Si el atributo de la clase tiene el mismo nombre que la etiqueta XML, podemos omitirlo. Tenemos que escribir esta anotación antes del método setter correspondiente. Mirar ejemplos en el siguiente subcapitulo.
- @XmlAttribute se utiliza para mapear atributos a los nodos XML.
- @XmlElementWrapper: ofrece la posibilidad de crear un contenedor alrededor de una representación XML. Este contenedor puede contener una colección de elementos.

El parámetro name = "nombre" permite especificar el nombre de la etiqueta XML.

Señala si la afirmación es verdadera o falsa:
Las anotaciones pueden declararse solo con parámetros.
a. Verdadero
b. Falso

Ejemplo de anotaciones

Tenemos la estructura XML siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<libreria>
          <ListaLibro>
               <Libro>
                    <autor>J.K.Rowling</autor>
                    <nombre>Harry Potter y la piedra filosofal</nombre>
                    <editorial>Salamandra</editorial>
                    <isbn>XXXX</isbn>
               <Libro>
          </ListaLibro >
          …….
          <lugar>XXXXXX</lugar>
          <nombre>Focus</nombre>
</libreria>
```

Para mapear la raiz es necesario una clase librería con anotación @XmlRootElement.

```
@XmlRootElement
              public class Libreria {
                     .....
              }
```

Esta clase tiene un atributo por cada etiqueta del XML incluido ListaLibro que es un array de elementos de tipo Libro:

```
@XmlElementWrapper(name = "ListaLibro")
@XmlElement(name = "Libro")
private ArrayList<Libro> ListaLibro;
```

La clase librería, es la clase que mapea el XML .

```
import java.util.ArrayList;
import javax.xml.bind.annotation.XmlElement;
import javax.xml.bind.annotation.XmlElementWrapper;
import javax.xml.bind.annotation.XmlRootElement;

//Esto significa que la clases "Libreria.java" es el elemento raiz
@XmlRootElement
public class Libreria {
    //Wrapper
    @XmlElementWrapper(name = "ListaLibro")
    @XmlElement(name = "Libro")
    private ArrayList<Libro> ListaLibro;
    private String nombre;
    private String lugar;
    public ArrayList<Libro> getListaLibro() {
        return ListaLibro;
    }
    public void setListaLibro(ArrayList<Libro> ListaLibro) {
        this.ListaLibro = ListaLibro;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public void setLugar(String lugar) {
        this.lugar = lugar;
    }
    public String getNombre() {
        return nombre;
    }
    public String getLugar() {
        return lugar;
    }
}
```

La clase libro que mapea cada uno de los nodos <Libro> del XML.

```
import javax.xml.bind.annotation.XmlType;


@XmlType(propOrder = { "autor", "nombre", "editorial", "isbn" })
public class Libro {
	private String nombre;
	private String autor;
	private String editorial;
	private String isbn;
	public String getNombre() {
		return nombre;
	}
	public String getAutor() {
		return autor;
	}
	public String getEditorial() {
		return editorial;
	}
	public String getIsbn() {
		return isbn;
	}
	public void setNombre(String nombre) {
		this.nombre = nombre;
	}
	public void setAutor(String autor) {
		this.autor = autor;
	}
	public void setEditorial(String editorial) {
		this.editorial = editorial;
	}
	public void setIsbn(String isbn) {
		this.isbn = isbn;
	}
}
```

Funcionamiento de JAXB

Para construir una aplicación JAXB necesitamos tener un esquema XML. Para obtener el esquema XML, seguimos los siguientes pasos para construir la aplicación JAXB:

1. Escribir el esquema: es un documento XML que contiene la estructura que se tomará como indicaciones para construir las clases. Estas indicaciones pueden ser, por ejemplo, el tipo primitivo al que se debe unir un valor de atributo en la clase generada.
2. Generar los ficheros fuente de Java: para esto usamos el compilador de esquema, ya que éste toma el esquema como entrada de información. Cuando se haya compilado el código fuente, podremos escribir una aplicación basada en las clases que resulten.
3. Construir el árbol de objetos Java: con nuestra aplicación, se genera el árbol de objetos java, también llamado árbol de contenido, que representa los datos XML que son validados con el esquema. Hay dos formas de hacer esto:
   - ✓ Instanciando las clases generadas.
   - ✓ Invocando al método unmarshall de una clase generada y pasarlo en el documento. El método unmarshall toma un documento XML válido y construye una representación de árbol de objetos.
4. Acceder al árbol de contenido usando nuestra aplicación: ahora podemos acceder al árbol de contenido y modificar sus datos.
5. Generar un documento XML desde el árbol de contenido. Para poder hacerlo tenemos que invocar al método marshall sobre el objeto raíz del árbol.

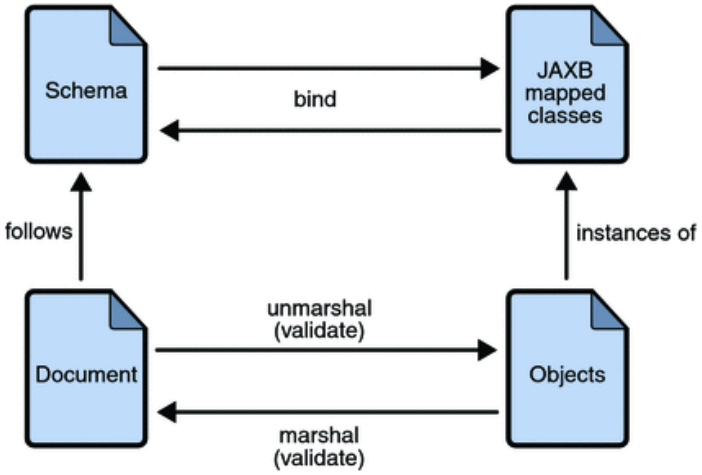
JAXB proporciona dos principales características:

- La capacidad de serializar (marshalling) objetos Java a XML.
- Lo inverso, es decir, deserializar (unmarshalling) XML a objetos Java.

O sea que JAXB permite almacenar y recuperar datos en memoria en cualquier formato XML, sin la necesidad de implementar un conjunto específico de rutinas XML de carga y salvaguarda para la estructura de clases del programa.

El compilador de JAXB (schema compiler) permite generar una serie de clases Java que podrán ser llamadas desde nuestras aplicaciones a través de métodos sets y gets para obtener o establecer los datos de un documento XML.

Los pasos en el proceso de enlace de JAXB están representados en la figura:



1. Generar clases: podemos escribirlas manualmente, como en el capitulo anterior o utilizar las clases generadas por el IDE JAXB automáticamente utilizando el esquema .xsd correspondiente al XML, lo veremos más adelante.
2. Deben compilarse todas las clases generadas, los archivos fuente y el código de la aplicación .
3. Aplicar Marshall a los objetos de las clases mapeadas para crear XML
4. Aplicar UnMarshall al XML para obtener objetos de las clases mapeadas .

❖ Marshall

Para poder hacer esta traducción, lo primero que necesitamos es un contexto, el cual instanciaremos pasándole la clase de nuestro root:

```
JAXBContext jaxbContext = JAXBContext.newInstance(Libreria.class);
```

Ya tenemos un contexto y ahora toca generar nuestro Marshaller

```
Marshaller marshaller = jaxbContext.createMarshaller();
marshaller.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
```

Para obtener un resultado, debemos pasarle una salida a nuestro marshaller. Aquí tienes un par de ejemplo de como hacerlo.

```
marshaller.marshal(libreria, System.out);
```

O bien,

```
File libreria-jaxb = this.getFile();
if (libreria-jaxbl != null) {
        marshaller.marshal(libreria, libreria-jaxb);}
```

❖ UnMarshall

Otra parte también fácil, tenemos varios inputs y utilizaremos el fichero por ahora para poder parsear. Partiendo de que tenemos ya un contexto, el cual reutilizaremos en esta parte.

```
Unmarshaller unmarshaller = jaxbContext.createUnmarshaller();
Libreria libreria = (Libreria) unmarshaller.unmarshal( libreria-jaxb);
```

Así de simple, recuperaremos o pasaremos de nuestro fichero xml a un objeto el cual tenemos ya mapeado.

Enlaces de interés para profundizar más en la materia:

https://www.javacodegeeks.com/2014/12/jaxb-tutorial-xml-binding.html#marshal

❖ Ejemplo: Marshall, Unmarshall

Una vez que tenemos el proyecto con las clases del apartado “Ejemplo de anotaciones”, vamos a añadir el siguiente programa

```
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.JAXBException;
import javax.xml.bind.Marshaller;
import javax.xml.bind.Unmarshaller;
public class JavaJAXB {
    private static final String LIBRERIA_XML = "./libreria-jaxb.xml";
    public static void main(String[] args) throws JAXBException, IOException {
        // Lista de LIbros
```

```
        ArrayList<Libro> libroLista = new ArrayList<Libro>();
        // Creamos varios libros
        Libro libro1 = new Libro();
        libro1.setIsbn("978-0060554736");
        libro1.setNombre("The Game");
        libro1.setAutor("Neil Strauss");
        libro1.setEditorial("Harpercollins");
        libroLista.add(libro1);
        Libro libro2 = new Libro();
        libro2.setIsbn("978-3832180577");
        libro2.setNombre("Feuchtgebiete");
        libro2.setAutor("Charlotte Roche");
        libro2.setEditorial("Dumont Buchverlag");
        libroLista.add(libro2);
        // Se crea La libreria y se le asigna la lista de libros
        Libreria libreria = new Libreria();
        libreria.setNombre("LIbreria sin limite");
        libreria.setLugar("Barrio Obrero");
        libreria.setListaLibro(libroLista);
        // Creamos un contexto de la clase JAXB y lo intanciamos
       JAXBContext context = JAXBContext.newInstance(Libreria.class);
       Marshaller m = context.createMarshaller();
       m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, Boolean.TRUE);
       // Lo creamos con system out
       m.marshal(libreria, System.out);
       // Escribimos en el archivo
       m.marshal(libreria, new File(LIBRERIA_XML));
       // Obtenemos las variables obtenidas del XML creado anteriormente
       System.out.println();
       System.out.println("Salida del XML: ");
       Unmarshaller um = context.createUnmarshaller();
       Libreria libreria2 = (Libreria) um.unmarshal(new FileReader(LIBRERIA_XML));
       ArrayList<Libro> lista = libreria2.getListaLibro();
       for (Libro libro : lista) {
       System.out.println("Libro: " + libro.getNombre() + " de " + libro.getAutor());
   }}}
```

Librerías para la conversión de documentos XML a otros formatos

En la mayoría de aplicaciones informáticas, hay que mostrar la información resultante de los procesos que se ejecutan, sobre todo en aplicaciones que generan información que implica tomar decisiones comerciales. Dicha información está almacenada normalmente en bases de datos o en archivos.

Hoy en día, XML está muy extendido, y muchas empresas guardan la información en ficheros o bases de datos con ese formato.

Hay muchos productos o herramientas informáticas que permiten convertir documentos XML a otros formatos.

En nuestro caso, vamos a optar por una herramienta que permite generar informes de todo tipo en Java de una forma sencilla: JasperReport.

Esta herramienta permite generar informes electrónicos en formato pdf, quizás el formato más usado debido a su portabilidad entre sistemas conservando la apariencia. Pero existen muchos más: xls, html, rtf, csv, xml, etc.

Señala si la afirmación es verdadera o falsa:
JasperReport permite generar documentos en formato html.
a. Verdadero
b. Falso

Introducción a JasperReport

En Java, durante un tiempo, la generación de informes fue uno de los puntos débiles del lenguaje, pero hoy en día, existen muchas librerías y herramientas dedicadas (varias de ellas, de código abierto) para la rápida generación de informes. JasperReports, es una de las más conocidas.

JasperReports es una herramienta que consta de un poderoso motor para la generación de informes. Está empaquetada en un archivo JAR y puede ser utilizada como una librería, la cuál podemos integrar en cualquier IDE de desarrollo en Java para desarrollar nuestras aplicaciones. Está escrita totalmente en Java, su código es abierto y es totalmente gratuita bajo los términos de la licencia GPL (Licencia Pública General).

Si visitas el siguiente enlace podrás acceder a la página de descarga de JasperReports para todas las plataformas. Encontrarás una lista con todas las versiones disponibles, no es necesario identificarte para poder bajarte la que desees.

Zona de descarga de JasperReports
(https://sourceforge.net/projects/jasperreports/files/jasperreports/)

Al descomprimir el fichero de la descarga, en el mismo hay varios directorios o carpetas. Comentamos brevemente qué contiene cada una:

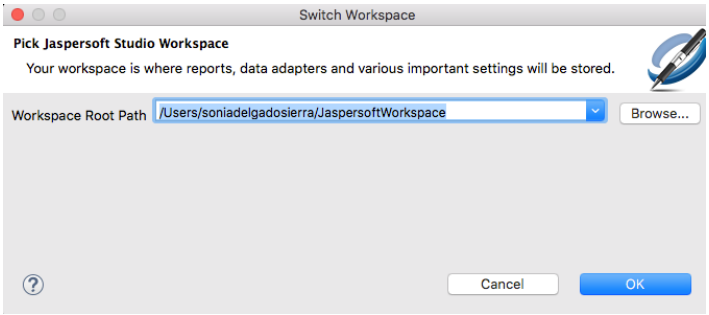
- build: es la librería JasperReports sin empaquetar, con todas las clases que incluye.
- demo: podemos encontrar algunos ejemplos de utilización de la librería. Estos ejemplos están preparados para ser compilados con la herramienta "ant". Puedes inspeccionar el código Java e intentar compilarlos y ejecutarlos.
- dist: es donde se encuentra realmente la librería empaquetada en un fichero JAR (jasperreports-3.7.4.jar) y algunos ficheros JAR que no utilizaremos. También podemos acceder a la documentación tipo javadoc.
- docs: es la referencia rápida en formato XML.
- lib: Diferentes librerías necesarias por JasperReports, como algunas para exportar a distintos formatos, para incluir gráficos, etc.
- src: Ficheros fuente de la librería.

En este enlace tienes la documentación en línea de la API de JasperReports.
http://jasperreports.sourceforge.net/api/index.html

Cómo instalar Jaspersoft Studio 6.21.3

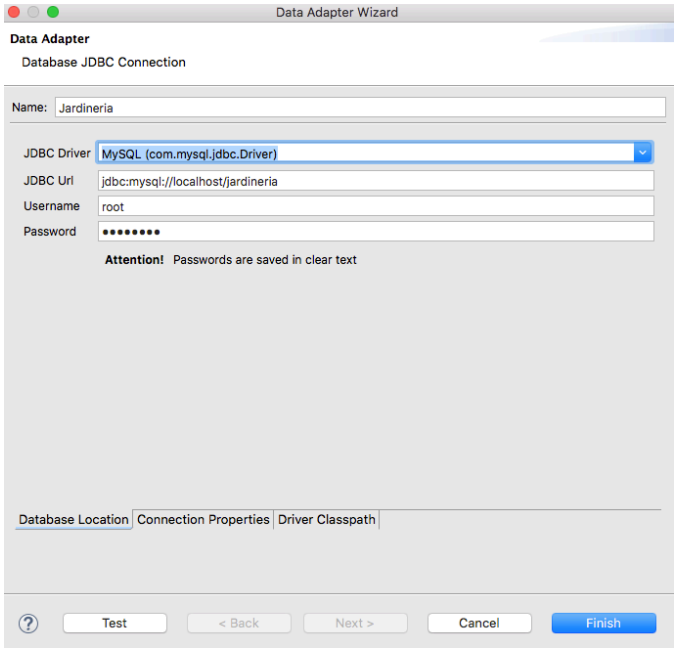
Para realizar informes en java vamos a utilizar la herramienta Jaspersoft Studio, podéis descargarla en cualquiera de estos enlaces: https://www.filehorse.com/es/download-jaspersoft-studio/91109/ ó en https://community.jaspersoft.com/project/jaspersoft-studio

Al instalarla os pedirá que seleccionéis el workspace de trabajo, es decir, una carpeta dentro de vuestro PC donde se almacenarán todos los reports que creéis, los data adapters y las configuraciones que realicéis. Esta carpeta se podrá cambiar más adelante.

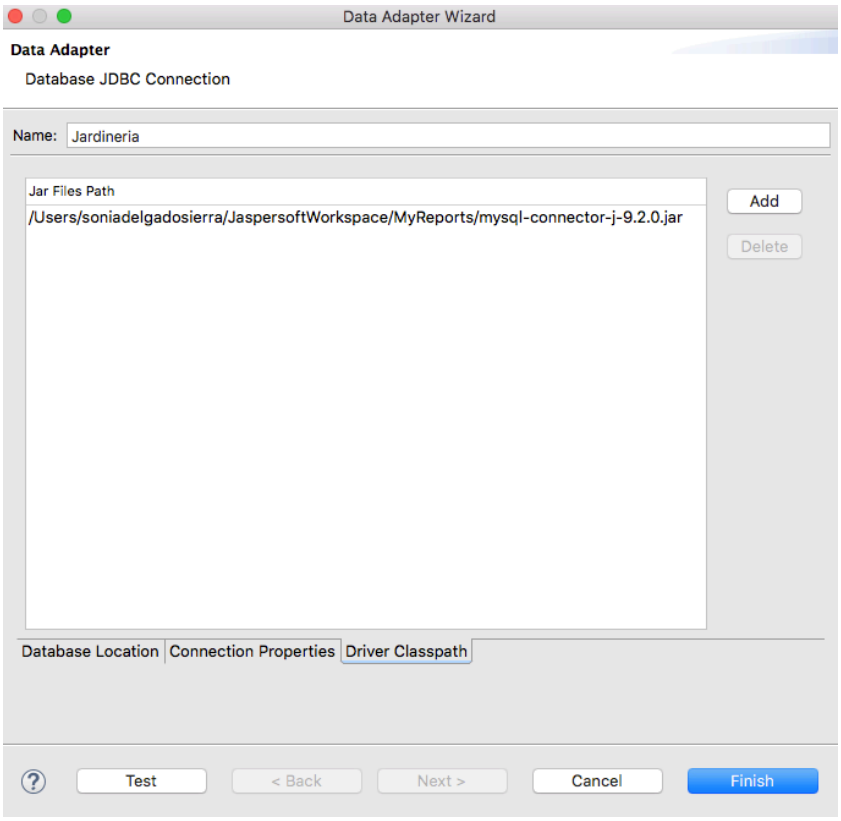


Una vez instalada la herramienta podéis empezar a crear vuestros propios **reports** aunque primero tenéis que crear vuestro **Adapter**.

Para crear el **Adapter**:
New Adapter -> Database JDBC Connection -> Le dais un nombre, el que queráis p.e: como vamos a usar la base de datos jardinería pues le ponemos `jardineria`, seleccionáis el driver de MySQL (com.mysql.jdbc.Driver) y rellenáis vuestra cadena de conexión con vuestra url, usuario y password.



NOTA: A veces es necesario en la carpeta classpath seleccionar el conector de JDBC de mysql (os bajáis cualquier versión, p.e: mysql-connector-j-9.2.0.jar, y lo añadís al Driver Path) .



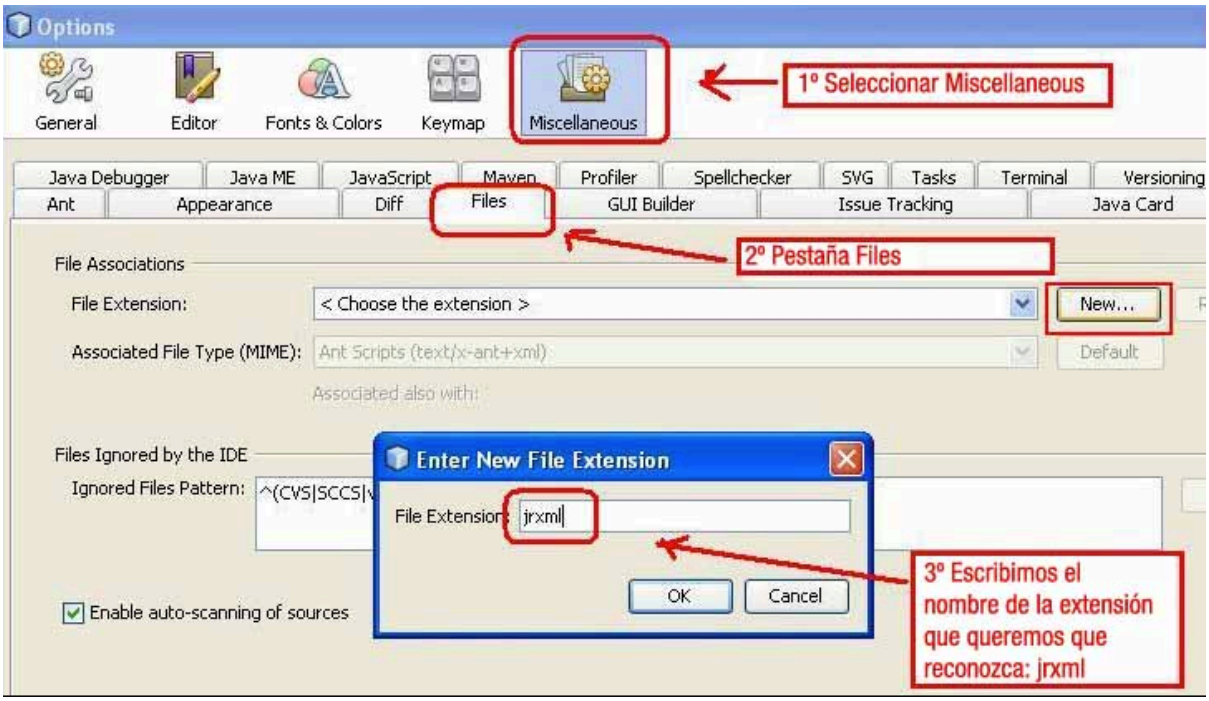
Para finalizar probár con el botón **test** si la conexión tiene éxito. Si pasa el test entonces le dais al botón de **Finish** y listo.
Ya podéis crear reports!!

Diseñar y compilar la plantilla

Las plantillas de los informes de JasperReports son sencillamente ficheros XML con la extensión .jrxml.

En el caso de Netbeans

Podemos hacer que NetBeans reconozca este tipo de ficheros como XML, para que cuando los editemos en el editor se muestren los mismos códigos de colores en las etiquetas y demás elementos de la sintaxis de XML.



En la imagen se ilustra cómo conseguirlo: en NetBeans pinchamos en el menú Tools, y ahí en Options. Ahí seleccionamos Miscellaneous, luego la pestaña Files. Entonces pulsamos en el botón New... para añadir la nueva extensión.

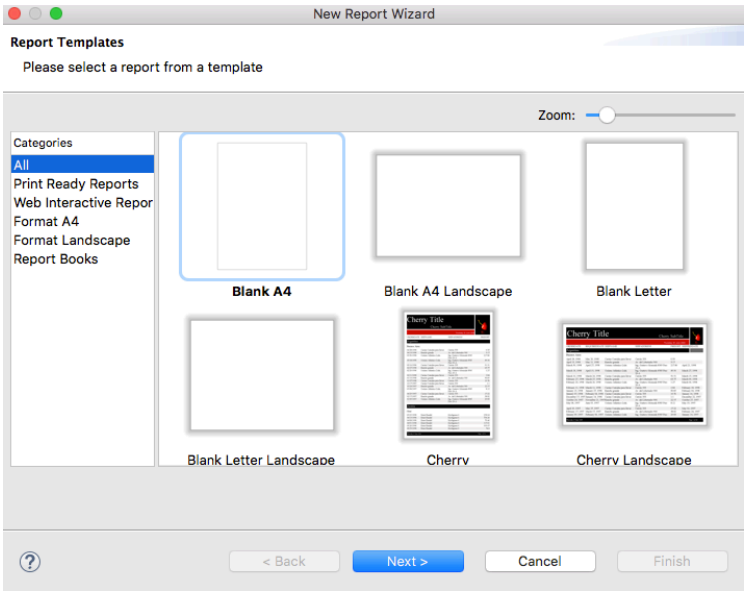
En el caso de VsCode

Una vez creados los informes/reports lo único que tendremos que hacer es llevar los reports creados a nuestra aplicación Java (p.e: dejarlos en la carpeta reportes), adjuntar en nuestro proyecto java (en java proyects la librería o librerías correspondientes (p.e: la de mysql y la de jaspereports si nuestro report trabaja con una bases de datos). Y luego en nuestra clase compilaremos, rellenaremos y visualizaremos el informe (report).

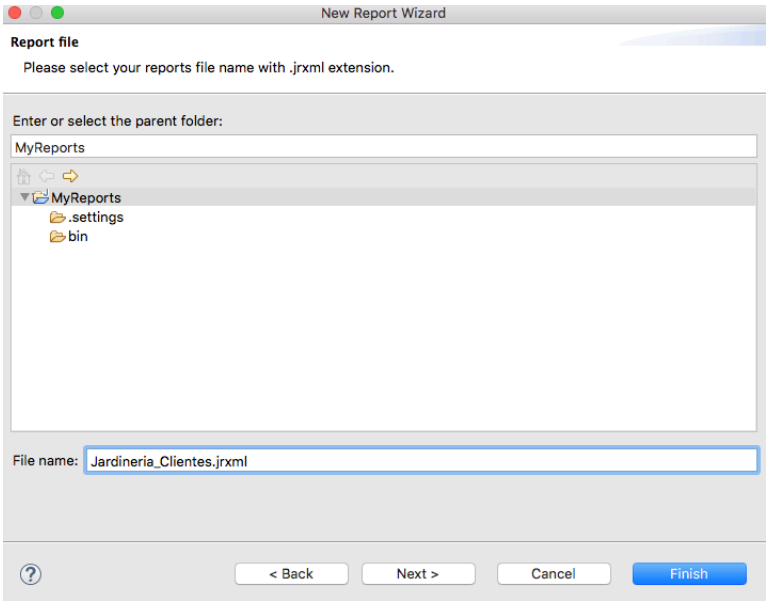
NOTA IMPORTANTE: OJO!! Si el informe .jrxml tiene logos/imágenes!! Si luego se quiere llevar ese informe/report a otro PC no se pondrá la ruta a la imagen como tal dentro del informe .jrxml, sino que habrá que añadirlo a la carpeta images de nuestro proyecto y al crear el informe .jrxml la ruta de la imagen será una expresión a nuestra imagen que estará en el classpath de nuestro IDE.

Los pasos a seguir para trabajar con JasperReport serían:

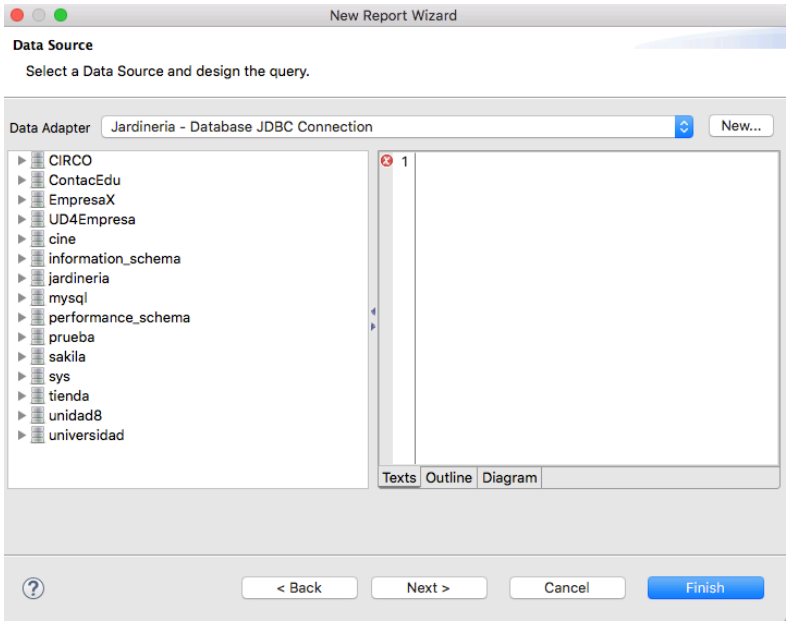
- Paso 1: Diseñar la plantilla del informe: un fichero .jrxml. Dentro del JasperSoft Studio una vez creado el Adapter se deberá crear la plantilla del informe a través del menú **File->New->Jaspert Report.**



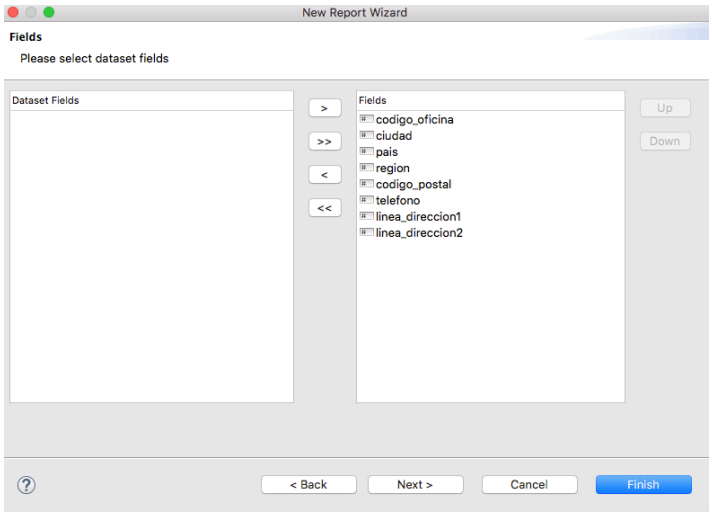
Aquí podrás seleccionar una plantilla ya prediseñada o crearlo tú desde cero.



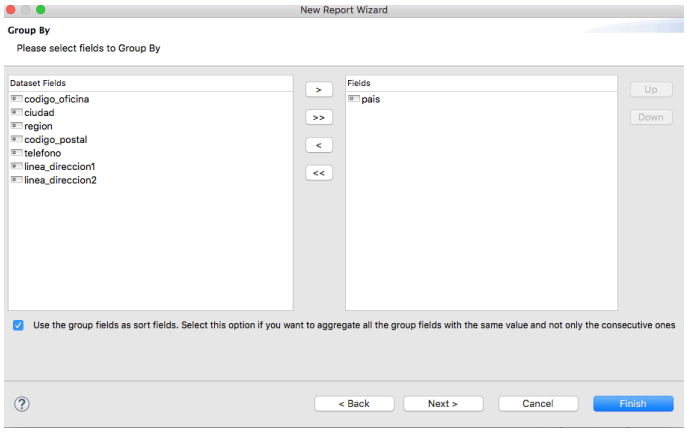
Aquí deberás seleccionar el nombre de tu report



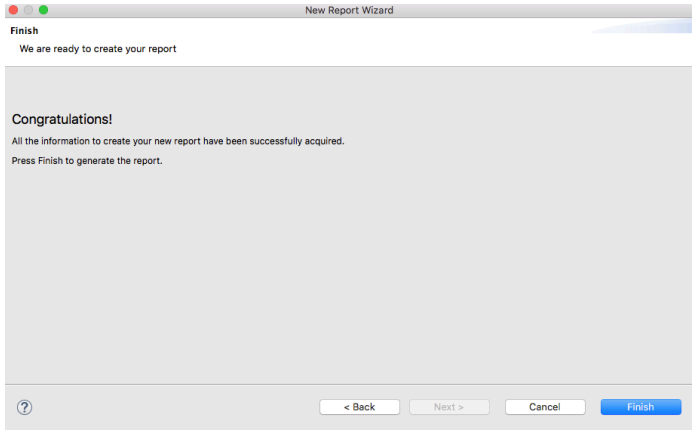
Aquí deberás seleccionar en el panel de la derecha la consulta SELECT que muestra todos los datos que quieres visualizar en tu report.
p.e: SELECT * FROM oficina



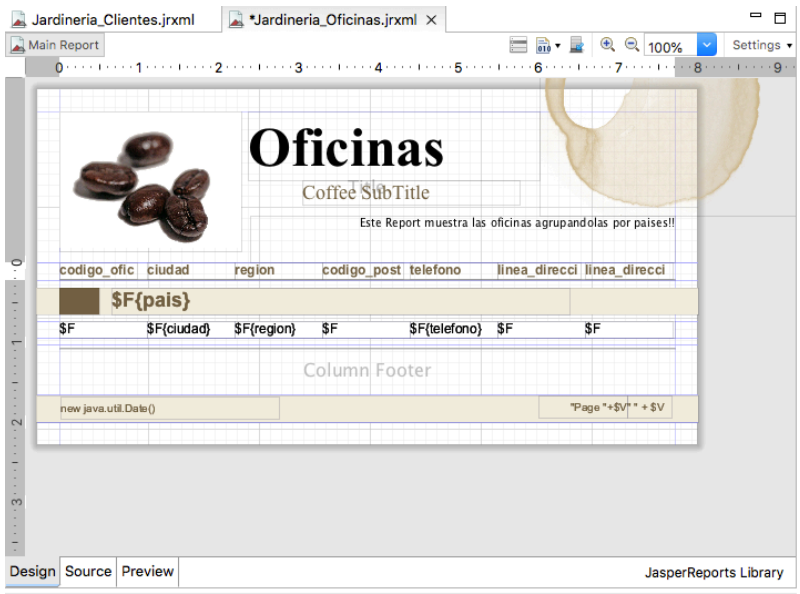
Aquí deberás seleccionar los campos (Fields) a mostrar en el report



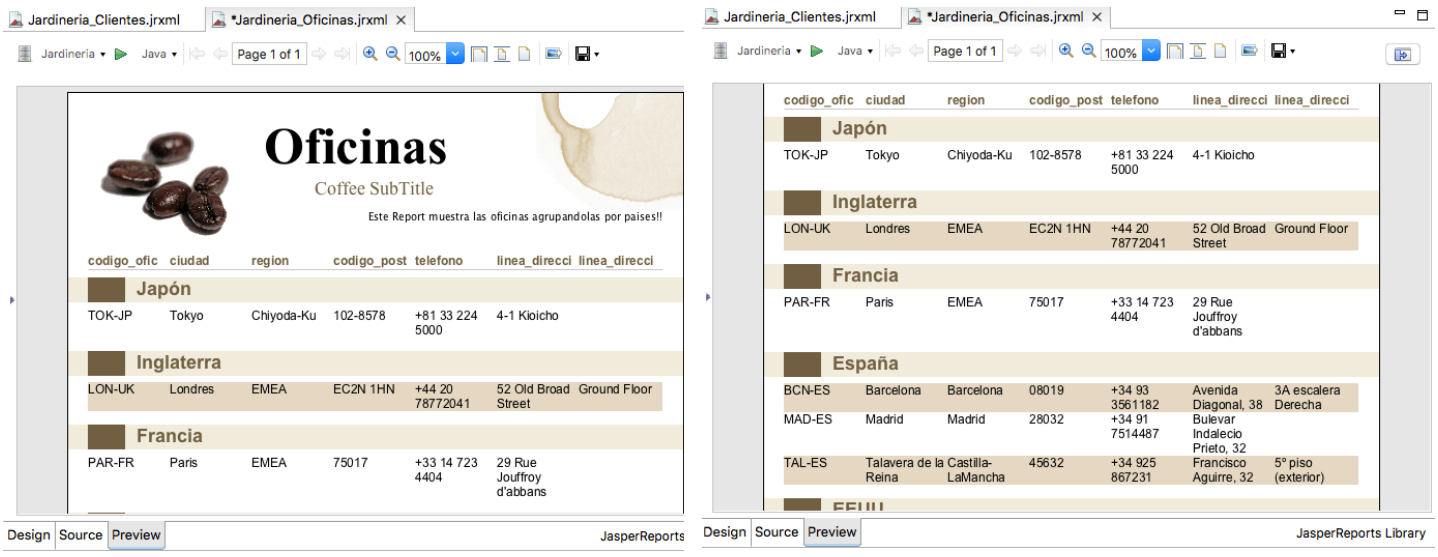
Si quieres agrupar por algún campo solo tienes que seleccionar el campo, en este caso vamos a sacar el report agrupado por país.



Pulsamos en el botón “Finish” y se muestra el diseño del report



Diseño (Design) del report



Vista preliminar (Preview) del report

El documento de diseño está representado por un archivo XML que mantiene la estructura de un archivo DTD (Document Type Definition) definido por el motor de JasperReports.

La generación de un diseño implica editar un archivo XML validado mediante:

```
<!DOCTYPE jasperReport PUBLIC "-//JasperReports//DTD Report Design//EN" "http://jasperreports.sourceforge.
net/dtds/jasperreport.dtd">
```

Estos documentos XML cuentan con una estructura similar a la de cualquier documento de texto. Fundamentalmente se siguen estas secciones:

- ✓ title Título del informe.
- ✓ pageHeader Encabezado del documento.
- ✓ columnHeader Encabezado de las columnas.
- ✓ detail Detalle del documento. Cuerpo
- ✓ columnFooter Pie de la columna.
- ✓ pageFooter Pie del documento.
- ✓ sumary Cierre del documento.

- Paso 2: Compilación: Una vez que se ha realizado el diseño, se compila antes de poder iniciar el proceso de carga de datos. La compilación se lleva a cabo a través del método compileReport().

En este proceso, el diseño se transforma en un objeto serializable de tipo net.sf.jasperreports.engine JasperReport, que luego se guarda en disco.

Señala si la afirmación es verdadera o falsa:
Los documentos XML que se usan en el diseño de los informes cuentan con una serie de secciones

a. Verdadero
b. Falso

Rellenar el informe con datos, exportar informe

Continuando con el apartado anterior…

- Paso 3: Rellenar el informe con datos:

Mediante los métodos fillReportXXX(), se puede realizar la carga de datos del informe, pasándole como parámetros el objeto de diseño (o bien, el archivo que lo representa en formato serializado) y la conexión JDBC a la base de datos desde donde se obtendrá la información que necesitemos.

Como resultado de este proceso, se obtiene un objeto que representa un documento listo para ser impreso, un objeto serializable de tipo JasperPrint. Este objeto puede guardarse en disco para su uso posterior, o bien puede ser impreso, enviado a la pantalla o transformado en PDF, XLS, CSV, etc.

- Paso 4: Visualización:

Ahora podemos optar por mostrar un informe por pantalla, imprimirlo, o bien obtenerlo en algún tipo específico de fichero, como PDF, etc.

- ✓ Para mostrar un informe por pantalla se utiliza la clase JasperViewer, la cual, a través de su método main(), recibe el informe a mostrar.
- ✓ Para imprimir el informe usaremos los métodos printReport(), printPage() o printPages(), contenidos en la clase JasperPrintManager.
- ✓ Para exportar los datos a un formato de archivo específico podemos utilizar los métodos exportReportXXX().

En el siguiente enlace se muestra un video de como intalar JasperReports con Netbeans.
https://www.youtube.com/watch?v=SvHWBFrLhPs

Nota: El principal inconveniente que puedes encontrarte al trabajar con JasperReports sin más, es sin duda el diseño del informe. Por ello, para facilitar el diseño de los mismos, y hacerlos de manera visual y cómoda se pueden usar otros productos como iReport, que es también una herramienta de software libre.

Hay mucha documentación sobre iReport en la red, aquí adjuntamos dos:

Tutorial iReport (https://adictosaltrabajo.com/2005/04/29/ireport/)
Crear informes con iReport (https://mundobyte.wordpress.com/2008/01/05/creacion-de-reportes-en-java-parte-i/)

EJERCICIOS BLOQUE 3

A partir del siguiente documento XML, https://learn.microsoft.com/es-es/dotnet/standard/linq/sample-xml-file-books

1. Muestra por pantalla los diferentes id de cada libro utilizando la librería DOM.
2. Muestra por pantalla una lista de autores y los títulos de sus libros.
3. Muestra por pantalla los títulos de los libros y sus precios. Ordena de más económico a más caro.
4. Muestra los libros por su genero
5. Traduce todas las etiquetas del XML y guardarlo en un fichero llamado libros.xml
    a. Catalog→Catalogo
    b. Book→Libro
    c. Title → Título
    d. Genre → Genero
    e. Price→Precio
    f. Public_date→Fecha de publicación
    g. Description → Descripción

Documentación de las aplicaciones desarrolladas

Una de las partes más importantes de la programación y en la que seguramente muchas veces nadie piensa es la documentación. Dejar el código bien documentado y comentado es una tarea igualmente importante y que en nuestra trayectoria laboral nos será de gran utilidad.

La manera más conocida de documentar el lenguaje Java es JavaDoc. Se trata de una utilidad de Oracle que permite documentar clases de Java.

En primer lugar, lo más común es usar comentarios. Los comentarios se pueden escribir de tres maneras:

```
//Comentario en una línea simple
/*
*Comentario en más de una línea
*/
/**
*Comentario con JavaDoc
*/
```

La estructura utilizada en JavaDoc es muy similar a la que se utiliza normalmente para comentarios, pero en la API se añade un asterisco extra al empezar, que es el característico para definirlo.

La gran peculiaridad de esta API es que permite añadir *tags* HTML dentro de los comentarios. Eso es debido a que JavaDoc, a través de los comentarios, genera un documento que servirá como documentación del proyecto.

Los comentarios para documentar, normalmente, se pueden poner en el código, encima de cualquier clase, en los métodos o atributos que queramos documentar.

Cuando se añadan comentarios, es necesario tener en cuenta que se debe describir cuál es la función de lo que se comenta. Por ejemplo, si comentamos una clase, haremos una breve explicación de lo que se encarga.

```
/**
 * Aquí pondremos un resumen de la finalidad de esta clase
 * Ej: Clase para documentar el funcionamiento de JavaDoc
 *
 * @author xxxx
 *
 */
public class JavaDoc {
    /**
     * Atributo nombre de la clase javaDoc
     */
    private String nombre;
    /**
     * <p>Ejemplo de Javadoc con tags html .
     * <a href="http://www.xxxx.es">Web xxxxx</a>
     * </p>
     * @param ejemplo String que pasamos por parametro
     * @return String con el resultado del metodo
     * @since 1.0
     */
    public String ejemploMetodoJavadoc(String ejemplo) {
    // Comentario con explicación de lo que se realiza en el  método
        return “OK”;
    }
}
```

Tags más importantes:

Tag	Descripción
@author	Indica el autor del desarrollo.
@deprecated	Indica que el método ó clase es obsoleto (propio de versiones. .anteiores) y no se recomienda su uso.
@param	Se usara para definir un parámetro de un método, es requerido para todos los parámetros del método.
@return	Se usa para indicar qué es lo que devuelve el método, no se usa para los métodos void.
@see	Asocia con otro método o clase.
@version	Se utiliza para definir la versión del módulo o la clase.

Para empezar, vamos a ver un ejemplo de cómo implementar JavaDoc en una clase.

Todos estos comentarios nos servirán para crear la documentación. Para generarla, podemos hacerlo desde la línea de comandos ejecutando este comando:

```
javadoc –d doc src\
```

También podremos hacerlo a través de nuestro IDE. Dependiendo del IDE utilizado accedemos a *Project* > *Generate JavaDoc*, nos aparecerá una ventana que nos permitirá elegir dónde queremos guardar nuestra documentación. Y ya lo tendremos creado.

PRÁCTICA FINAL 1

Crea un programa que genere la siguiente estructura de directorios:

- Directorio `ejercicios`
  - Crea el fichero `pizzas.xml` y rellénalo con diferentes pizzas junto con sus ingredientes. (Utiliza DOM). Por ejemplo:

```
<pizzas>
<pizza nombre="Margarita">
        <ingrediente>tomate</ingrediente>
        <ingrediente>queso</ingrediente>
</pizza>
<pizza nombre="Pepperoni">
        <ingrediente>tomate</ingrediente>
        <ingrediente>queso</ingrediente>
        <ingrediente>pepperoni</ingrediente>
</pizza>
</pizzas>
```

  - Lee el fichero `libros.xml` (supón que ya existe) y muestra únicamente los autores. (Utiliza DOM). A continuación se muestra el fichero:

```
<?xml version="1.0" encoding="UTF-8"?>
<libros>
    <libro>
        <titulo>El Quijote</titulo>
        <autor>Miguel de Cervantes</autor>
    </libro>
    <libro>
        <titulo>Cien Años de Soledad</titulo>
        <autor>Gabriel García Márquez</autor>
    </libro>
    <libro>
        <titulo>1984</titulo>
        <autor>George Orwell</autor>
    </libro>
    <libro>
        <titulo>Orgullo y Prejuicio</titulo>
        <autor>Jane Austen</autor>
    </libro>
    <libro>
        <titulo>El Señor de los Anillos</titulo>
        <autor>J.R.R. Tolkien</autor>
    </libro>
</libros>
```

- Directorio `alumnos`
  - Crea 3 alumnos (tendrás que crear una clase `Alumno`), guardando los objetos en ficheros para luego poder recuperarlo.
    - 1 alumno→1 fichero `(alumno1.dat, alumno2.dat y alumno3.dat)`

```
Los alumnos pueden ser estos:
Alumno alumno1 = new Alumno("Juan", 20);
Alumno alumno2 = new Alumno("María", 22);
Alumno alumno3 = new Alumno("Pedro", 21);
```

- Directorio `profesores`
  - Crea un fichero de texto con el listado de nombres de los profesores.

```
Los profesores pueden ser estos:
Arrays.asList("Profesor A", "Profesor B", "Profesor C")
```

PRÁCTICA FINAL 2

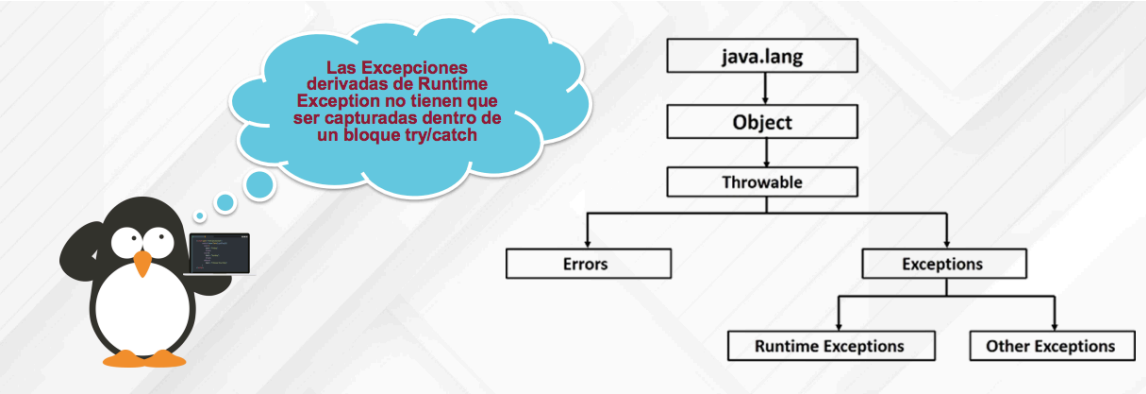
Realizar los siguientes programas JAVA referentes al manejo de ficheros:

1. Muestra la información de un fichero o de un directorio: nombre, ruta relativa y absoluta, si permite lectura y escritura y su tamaño.
2. Enumera todos los ficheros y subdirectorios que tiene un directorio dado.
3. Crea un directorio vacío en la ruta dada: C:\\Users\\Alumni\\Documents\\NetBeansProjects\\Ficheros\\src\\ficheros
4. Crea un fichero de texto de forma secuencial en el que vaya el siguiente texto: "Ejemplo de escritura en un fichero de texto o txt".
5. Visualiza el fichero de texto creado anteriormente.
6. Escribe un fichero binario Empleados (.dat) de manera secuencial. Con la siguiente información
   Departamento "Contabilidad","Informática","Dirección","Análisis","Finanzas","Hardware"
   Nº Empleados 3,10,2,5,4,8
7. Mostrar la información del fichero binario (.dat) de forma secuencial creado anteriormente.
8. Escribe un fichero binario alumnos (.dat) de forma aleatoria. Con la siguiente información.
   Apellido "FERNANDEZ","LOPEZ","GOMEZ","CHEN","SERRANO","CASILLAS", "ALONSO"
   Edad 17, 20, 18, 17, 19, 21, 20
   Nota 7.5, 4.2, 6.5, 8.0, 3.2, 9.2, 9.9
9. Visualiza el fichero binario alumnos (.dat) de forma aleatoria.
10. Convierte el fichero binario (.dat) alumnos a XML usando DOM.
11. Visualiza el fichero XML alumnos usando DOM.
12. Visualiza el fichero XML alumnos usando SAX.

Control de excepciones

Java tiene un sistema para controlar los errores que se pueden producir durante la ejecución, llamado control de excepciones .

Las excepciones en Java son objetos que derivan de las clase Exception y que a su vez derivan de la clase Throwable.



Se basa en que cuando se produce un error que se quiere controlar en ese punto, se lanza una excepción. Y en otro punto, se controla la excepción y se maneja el error en consecuencia.

Facilita el control de errores, ya que distintos tipos de excepciones se pueden controlar de la misma forma.

La obligación de control de excepciones garantiza un mejor control de los errores.

Sentencia try-catch-finally

Para controlar las excepciones, se utiliza la sentencia try-catch.
Dentro del bloque try, se ejecutan las sentencias sobre las que se quiere controlar las excepciones.
Dentro del bloque catch, se muestran las sentencias que se deben ejecutar en caso de error.

```
//syntax for try...catch block statement
try {
// code block to track for the exception
}
catch (Exception eObj) {
// exception handler for exception
}
finally {
// finally block code statement executes after try block
}
```

Como parámetro, el catch pone el tipo de excepción a controlar. Si se quieren controlar todas, se usa Exception.
La sentencia finally es opcional, y contiene las sentencias que se van a ejecutar exista o no una excepción.

Ejemplo Escribir en Fichero con control de excepciones:

```
try {
          FileOutputStream fileOutputStream = new FileOutputStream(copia, false);
          for (Integer b : bytes) {
                    fileOutputStream.write(b);
          }
          fileOutputStream.close();
} catch (IOException ex) {
          System.out.println("No se ha podido escribir en el fichero");
} finally {
          System.out.println("Función fileOutputStream finalizada");
}
```

Si las excepciones no se controlan, se muestra el detalle por consola. Muestra la pila de ejecución.

